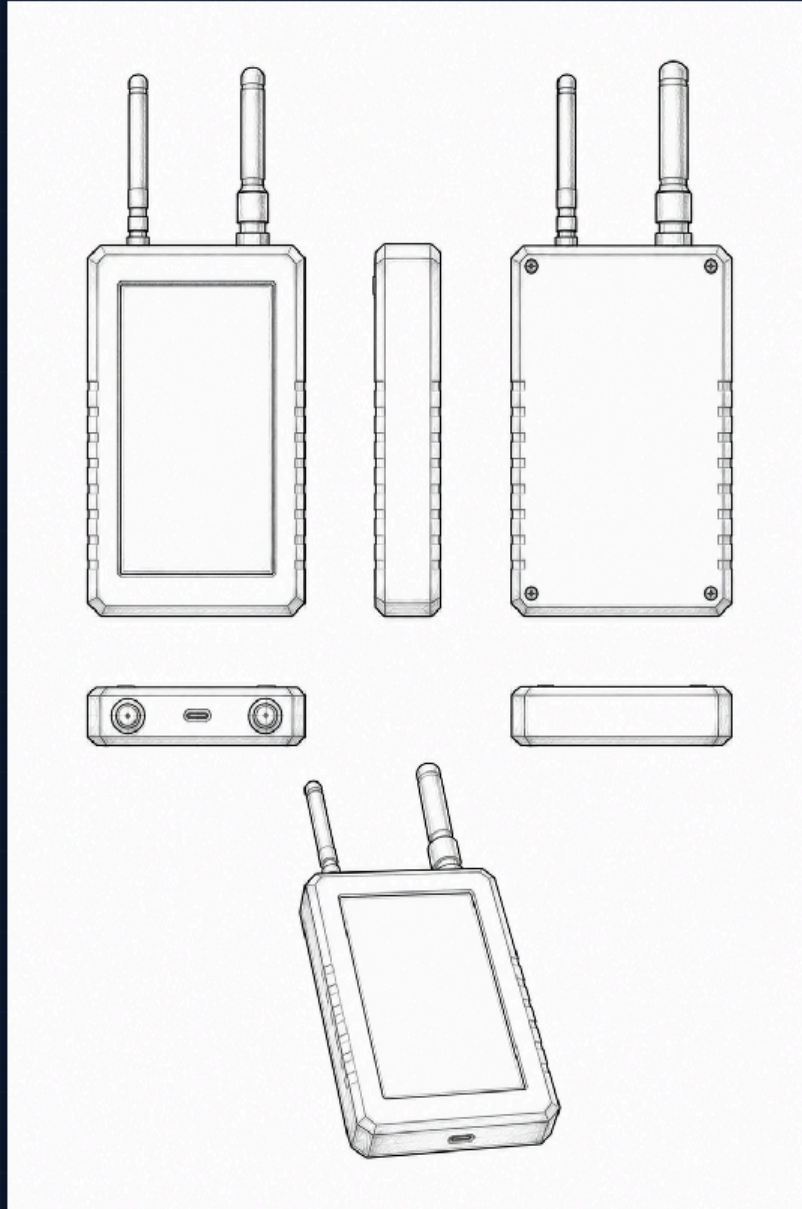


COMMUNITY GUIDE

ESP32 MARAUDER



HARDWARE REFERENCE · MULTI-VIEW TECHNICAL ILLUSTRATION

FOR EDUCATIONAL & AUTHORIZED USE ONLY

Written & compiled by

Runaque

MADE IN ANTWERP

github.com/Runaque

ESP32 Marauder

Ultimate Community Guide

Feature Reference • Practical Tier List • Beginner's Handbook

For Educational, Defensive & Authorized Testing Purposes Only

Written & Compiled by

Runaque

Community Guide — ESP32 Marauder Project by JustCallMeKoko

github.com/Runaque

Absolute Beginner Prerequisites

This guide covers a lot of ground — from unboxing your first Marauder device all the way through to compiling custom firmware and building a ruggedised War-Box. Some of that content assumes familiarity with tools and concepts that experienced makers take for granted but that are genuinely unfamiliar to someone just starting out.

This chapter exists to bridge that gap. If you have never touched a command line, never installed a driver, never opened a terminal, or never heard the word 'serial port' before — this is your starting point. Read it before anything else. It will make every other section in this guide make sense.

If you are already comfortable with computers beyond basic use, feel free to skim or skip ahead. Nothing here will be new to you.

1. What Is a Terminal / Command Line?

Most people interact with their computer through a graphical interface — clicking icons, opening windows, dragging files. A terminal (also called a command line, command prompt, or console) is a text-based alternative way to communicate with your computer. Instead of clicking, you type instructions and press Enter.

It sounds intimidating but the reality is that most of what you will do in a terminal throughout this guide consists of typing one or two commands from a clearly written example, pressing Enter, and reading the output. You are not expected to memorise anything or invent commands — just follow the instructions.

How to Open a Terminal

- Windows: Press the Windows key, type 'cmd' or 'PowerShell', and press Enter. A black or blue window with a text cursor will open. This is your command prompt.
- macOS: Press Command + Space to open Spotlight, type 'Terminal', and press Enter. A window with a text prompt will open.
- Linux: Press Ctrl + Alt + T on most distributions, or search for 'Terminal' in your application menu.

 **Tip:** The terminal looks more intimidating than it is. The vast majority of commands in this guide are a single line you can copy directly from the text. If something goes wrong, the terminal will tell you — and the error message is usually a good starting point for figuring out what to fix.

2. What Is a COM Port / Serial Port?

When you plug a Marauder device (or any ESP32) into your computer via USB, the computer creates a virtual serial port to communicate with it. On Windows this appears as a COM port — you might see it listed as COM3, COM5, or another number. On macOS and Linux it appears as a file path like `/dev/ttyUSB0` or `/dev/tty.usbserial-XXXX`.

This serial port is how firmware flashing tools (like the web flasher and ESPTool) talk to the device. It is also how Marauder's CLI communicates with your computer when you connect a serial terminal.

How to Find Your COM Port on Windows

- Right-click the Start button and choose Device Manager.
- Expand the 'Ports (COM & LPT)' section.
- Plug in your Marauder device. A new entry should appear — this is your COM port. Note the number (e.g. COM5).
- If nothing appears, your cable may be charge-only (see the Wall of Shame section) or you may need to install a driver (see section 4 below).

How to Find Your Serial Port on macOS / Linux

- Open a terminal and type: `ls /dev/tty*` — this lists all serial devices.
- Plug in your Marauder device, wait a few seconds, and run the command again.
- The new entry that appeared is your device's serial port. It will look something like `/dev/ttyUSB0` or `/dev/tty.usbserial-1420`.

3. What Is a Driver?

A driver is a small piece of software that tells your operating system how to communicate with a specific piece of hardware. When you plug in a USB device, Windows (or macOS or Linux) looks for a matching driver. If it finds one, the device works. If it does not, the device is listed as 'Unknown Device' and nothing happens.

Most ESP32 development boards use one of two USB-to-serial chips to bridge the USB connection to the ESP32's UART interface: the CP2102 (made by Silicon Labs) or the CH340/CH341 (made by WCH). Both require a driver on Windows and sometimes on macOS. Linux usually has both built in.

How to Install the Right Driver

- CP2102 driver: Search 'Silicon Labs CP210x driver' and download from the official Silicon Labs website (silabs.com). Install it and restart if prompted.
- CH340 / CH341 driver: Search 'CH340 driver' and download from a reputable source. The WCH official website (wch-ic.com) hosts the official version.
- Not sure which chip your board uses? Look at the small chip near the USB port on your ESP32 board. The chip will have text printed on it — look for 'CP2102', 'CH340', or 'CH341'. If you cannot read it, try both drivers — only the correct one will work and installing an extra driver does no harm.

 **Tip:** After installing a driver, unplug and replug the device. Then check Device Manager again. If the COM port now appears without a warning triangle, the driver is installed correctly.

4. What Is Python and Why Do You Need It?

Python is a free, widely used programming language. You do not need to know how to write Python to use this guide. The only reason it comes up is that one of the firmware flashing tools — ESPTool — is a Python-based command-line utility, meaning Python needs to be installed on your computer before ESPTool will work.

How to Install Python

- Windows: Go to python.org, click 'Download Python', and run the installer. On the first screen of the installer, tick the box that says 'Add Python to PATH' before clicking Install. This is important — without it, Python commands will not work in the terminal.
- macOS: Python 3 is available via the official python.org installer, or via Homebrew (brew install python3) if you have Homebrew installed.
- Linux: Python 3 is almost certainly already installed. Confirm by typing `python3 --version` in a terminal.

Once Python is installed, open a terminal and type: `python --version` (or `python3 --version` on macOS/Linux). If you see a version number like 'Python 3.11.2', Python is working correctly.

5. What Is pip?

pip is Python's package manager — a tool that downloads and installs Python libraries and utilities. It comes with Python and is how you install ESPTool. When this guide says 'run pip install esptool', it means: open your terminal and type that command. pip will download ESPTool from the internet and install it automatically.

You do not need to understand what pip is doing internally. Think of it like an app store for command-line tools — you tell it what you want installed and it handles the rest.

6. What Is USB vs UART?

These two terms describe different ways of connecting devices together and come up frequently in ESP32 discussions.

USB (Universal Serial Bus)

USB is the standard connector and protocol on your computer — the rectangular ports you plug cables into. When an ESP32 board has a USB port that your computer recognises as a serial device (and shows as a COM port), the board contains a USB-to-serial chip (CP2102 or CH340) that converts USB signals into the UART signals the ESP32 understands. This is the convenient, plug-and-play setup used by most quality boards.

UART (Universal Asynchronous Receiver-Transmitter)

UART is a simpler, older serial communication standard that uses two wires: TX (transmit) and RX (receive). The ESP32 communicates natively over UART — but your computer does not have UART ports. The USB-to-serial chip on a quality board bridges this gap invisibly.

On clone devices that omit the USB-to-serial chip, you are left with raw UART pins or pads. To flash firmware on such a device, you need a separate USB-to-UART adapter (sometimes called a USB-TTL adapter or FTDI adapter) — a small dongle that plugs into your computer's USB port and has TX, RX, GND, and VCC pins you connect to the device's matching pads.

💡 **Tip:** A USB-to-UART adapter costs around €3-5 on AliExpress and is a useful tool to have regardless. If you end up with a clone device that lacks a proper USB port, an adapter is your recovery path for firmware flashing.

7. What Is PlatformIO?

PlatformIO is a development environment for embedded systems — meaning it is a tool for writing, compiling, and flashing software onto microcontrollers like the ESP32. It runs as an extension inside Visual Studio Code (a free code editor from Microsoft) and handles the complex parts of embedded development automatically: downloading the right compiler, managing libraries, and configuring the build for your specific hardware target.

You only need PlatformIO if you want to compile Marauder from source — for example, to add a custom boot logo or colour theme. For standard firmware flashing using pre-built binaries, you do not need it at all. The web flasher handles standard installs with no development environment required.

How to Install PlatformIO

- Step 1 — Download and install Visual Studio Code from code.visualstudio.com (free, Windows/macOS/Linux).

- Step 2 — Open VS Code, click the Extensions icon in the left sidebar (it looks like four squares), and search for 'PlatformIO IDE'.
- Step 3 — Click Install. PlatformIO will install itself and its dependencies — this takes a few minutes.
- Step 4 — Restart VS Code when prompted. PlatformIO is now ready to use.

 **Tip:** PlatformIO's first build of a project downloads the ESP32 toolchain and all required libraries automatically. This can take 5-10 minutes on a slow connection but only happens once — subsequent builds are much faster.

8. What Is an SD Card and Why Does Formatting Matter?

An SD card (Secure Digital card) is the small rectangular flash storage card that slots into your Marauder device to store captured data. You have almost certainly used one in a camera or phone.

What matters for Marauder is the file system the card is formatted with. A file system is the organisational structure that determines how files are stored and named on the card. Windows computers typically offer FAT32, exFAT, and NTFS as options. Marauder requires FAT32 specifically — it cannot read exFAT or NTFS cards.

New cards larger than 32GB are usually formatted as exFAT by default. Even 32GB and smaller cards sometimes come formatted differently. Always format the card as FAT32 yourself before first use rather than assuming it is already correct.

9. What Does 'Flashing Firmware' Mean?

Firmware is the software that runs on a hardware device like the ESP32 — it controls everything the device does. 'Flashing' firmware means writing a new version of that software onto the device's internal memory, replacing whatever was there before.

When this guide talks about flashing a .bin file to your ESP32, it means: using a tool (the web flasher or ESPTool) to copy that binary file into the ESP32's flash memory over the USB connection. After flashing, the device reboots and runs the new firmware.

The .bin file is a compiled binary — a file containing the actual machine code instructions that the ESP32's processor executes. You do not read or edit .bin files directly. You simply download the correct one for your hardware and flash it.

10. What Is GitHub?

GitHub is a website where software developers share and collaborate on code. The official ESP32 Marauder source code, firmware releases, documentation, and issue tracker all live on GitHub at github.com/justcallmekoko/ESP32Marauder.

You do not need a GitHub account to download firmware releases or read documentation — those are publicly accessible. You only need an account if you want to report bugs (open an issue) or contribute to the project. Browsing GitHub feels unfamiliar at first but the layout is consistent: the main page shows the source code, the Releases tab (on the right side) shows downloadable firmware binaries, and the Issues tab shows known bugs and community discussions.

 **Tip:** When downloading firmware from the Releases page, look for the Assets section under each release. This is where the downloadable .bin files live. Read the release description carefully before downloading — it explains which binary file corresponds to which hardware variant.

You Are Ready

If you have read through this section, you now have the conceptual foundation to make sense of every technical instruction in this guide. You do not need to have done any of this yet — just knowing what a COM port is, what a driver does, and what 'flashing firmware' means will prevent the confusion that stops most beginners in their tracks.

The rest of the guide assumes this foundation. When you hit a term that is not explained in context, come back here. Everything else builds on these basics.

Introduction

ESP32 Marauder is an open-source portable wireless analysis platform built around the ESP32 microcontroller. Designed by the developer known as JustCallMeKoko, it turns a small, inexpensive chip into a handheld tool capable of Wi-Fi and Bluetooth reconnaissance, packet capture, wardriving, wireless surveying, and educational security demonstrations — all without needing a laptop or expensive professional equipment.

Think of it as a pocket-sized window into the invisible world of wireless signals buzzing around you at all times. From coffee shops to your own home network, Marauder lets you see what devices are broadcasting, what networks are visible, and how the wireless environment is behaving — all in real time.

Who Uses ESP32 Marauder?

- Wireless security enthusiasts and hobbyists
- RF surveyors and wardrivers who map wireless environments
- Students learning Wi-Fi networking concepts hands-on
- Security professionals performing authorized assessments
- Hardware tinkerers experimenting with ESP32 capabilities

Two Ways to Get Started

Build Your Own

Many community members assemble their own Marauder using supported ESP32 development boards, small screens, GPS modules, batteries, and 3D-printed enclosures. This DIY route is flexible, customizable, and lower cost — ideal for users comfortable with hardware assembly and firmware flashing.

Buy a Prebuilt Device

Users who prefer a polished, ready-to-use experience can purchase official prebuilt Marauder hardware from the creator JustCallMeKoko or approved community vendors. Official devices come professionally assembled and preconfigured, delivering a streamlined out-of-box experience.

 **Tip:** Either route gives you access to the same firmware and feature set. Hardware differences mainly affect ergonomics, battery life, display size, and GPS support.

A Word on Chinese Clones & the Shadow Market

There exists a well-known shadow market of Marauder-inspired devices — primarily originating from Chinese manufacturers on platforms like AliExpress, Alibaba, and various reseller storefronts. These range from near-identical reverse-engineered copies of JustCallMeKoko's official hardware to custom PCB designs that replicate the feature set at a significantly lower price point.

If the official device is sold out or outside your budget, a clone may be tempting. This section exists so you go in with realistic expectations rather than frustrating surprises.

What Clones Sometimes Get Right

At their best, Chinese clone devices can be genuinely functional. The core ESP32 chip is the same, the display driver is often identical, and the firmware — being open source — runs the same on a clone as on an official device. Many hobbyists have had perfectly positive experiences with clone hardware for basic scanning, wardriving, and packet capture workflows.

- Core Wi-Fi and BLE scanning functions typically work as advertised
- The same firmware binaries often flash successfully if the hardware is wired compatibly
- Build quality on better clones can be acceptable for casual use

Where Clones Commonly Fall Short

Build quality is the most obvious concern. Clone devices are typically built to a price point, which means cheaper components, less careful soldering, thinner enclosures, lower-grade batteries, and display modules that may have shorter lifespans or more inconsistent colour reproduction. A clone that works fine on day one may develop faults within months of regular use.

The more critical issue for beginners is firmware flashing access. On JustCallMeKoko's official hardware and on quality DIY builds, the device exposes a standard USB port that your computer recognises as a serial device — making firmware updates straightforward via the web flasher or ESPTool.

Many clone devices cut costs by omitting the USB-to-serial chip entirely, replacing the USB port with a bare UART port, exposed UART contact pads on the PCB, or no accessible flashing interface at all. This means:

- You may not be able to update the firmware without purchasing a separate UART-to-USB adapter and soldering or clamping to exposed pads
- Flashing alternative firmware variants — such as Bruce firmware or HaleHound — may be difficult or impossible without hardware modification
- If the clone ships with outdated or modified firmware, you may be stuck with it

- Recovery from a bad flash becomes significantly more complicated without direct USB access

Missing or Modified Features

Some clone PCBs omit components present on the official hardware — GPS module connectors, SD card slots, or tactile buttons may be absent, repositioned, or wired differently. A clone advertised as 'full featured' may be missing the GPS header entirely, making wardriving impossible without hardware modification.

Firmware built specifically for official hardware variants may also behave incorrectly on a clone if the pin assignments differ. A display that works perfectly on the official device may show a blank screen or garbled output on a clone using a slightly different wiring layout, even with an identical-looking enclosure.

Alternative Firmware Compatibility

The Marauder ecosystem has grown to include alternative firmware projects — most notably Bruce firmware and HaleHound — which extend or replace Marauder's feature set with different capabilities and interfaces. Flashing these alternatives requires reliable USB serial access. On a clone device without a proper USB port, exploring the broader firmware ecosystem becomes a hardware project before it becomes a software one.

The Bottom Line on Clones

If budget is a genuine constraint and the official device is unavailable, a clone can be a reasonable starting point — with the understanding that you are accepting trade-offs in build quality, longevity, and firmware flexibility. Before purchasing any clone, search the community Discord and forums specifically for that device model. If other users have successfully flashed firmware and confirmed all hardware features work, it is a much safer bet than buying blind.

⚠ Never purchase a clone device from an unknown seller claiming it is an 'official' JustCallMeKoko product. Official hardware is sold exclusively through JustCallMeKoko's own channels and verified vendors. Counterfeit claims are common in this market.

💡 Tip: If you are buying a clone specifically to experiment with firmware flashing and alternative builds, verify before purchasing that the device exposes a USB serial port or clearly documented UART pads — and confirm that the community has successfully flashed it before. A device you cannot reflash is a device you cannot recover if something goes wrong.

Legal & Ethical Notice

⚠ Many Marauder functions can violate laws, regulations, terms of service, or radio-use rules if used against networks or devices without authorization. Actions such as deauthentication, spoofing, disruption, and packet interception may expose users to civil or criminal liability depending on jurisdiction.

Core Rules

- Only test against networks and devices you own or are explicitly authorized to assess.
- Never interfere with public, corporate, or third-party networks.
- Passive sniffing may also be regulated in some jurisdictions — research your local laws.
- Radio laws vary significantly by country.

This guide is written for educational, defensive, and authorized testing purposes only.

Ethics in Cybersecurity

Before diving into what Marauder can do, it is worth taking a moment to think about why ethics matter in this space — and what kind of person you want to be when you pick up this tool.

Cybersecurity is one of the few fields where the same knowledge, the same skills, and often the same exact tools can be used to protect people or to harm them. The difference is never the tool. It is always the person holding it and the choices they make.

The Hat Spectrum

The security community has long used a colour-coded shorthand to describe the motivations and ethics of practitioners. It comes from old Western films — the heroes wore white hats, the villains wore black ones. The model has stuck because it maps surprisingly well onto reality.

White Hat — The Defender

White hat practitioners use their skills to protect systems, educate others, and strengthen security with full authorisation from the people who own the systems they work on. They may do everything a black hat does technically — scanning networks, capturing packets, probing for weaknesses — but they do it with permission, with transparency, and with the goal of making things safer.

White hats work as penetration testers, security researchers, network engineers, and educators. They disclose vulnerabilities responsibly, document their findings, and leave systems more secure than they found them. This guide, and the intent behind sharing it, sits firmly in white hat territory.

Grey Hat — The Ambiguous Middle

Grey hats occupy uncomfortable ground. They often have good intentions — finding vulnerabilities in systems they do not own, then notifying the owners rather than exploiting the weakness. But the road to that good outcome still ran through unauthorised access, which is illegal regardless of intent in most jurisdictions.

Grey hat activity is a reminder that good intentions do not override the law, and that 'I was just looking' is not a legal defence. Many grey hats eventually move white — joining bug bounty

programmes, getting authorisation before testing, and finding legitimate channels for their curiosity.

Black Hat — The Attacker

Black hats exploit systems, networks, and people for personal gain, disruption, or malice — without authorisation and without regard for the harm they cause. This includes stealing credentials, disrupting services, spying on communications, deploying ransomware, and countless other forms of digital harm.

The skills involved are often identical to those of a white hat. What separates them is consent, intent, and the willingness to cause harm to real people — individuals who lose data, businesses that lose money, hospitals that lose access to critical systems.

Why We Stay on the White Side

There are practical reasons and principled reasons, and both matter.

The Practical Case

Unauthorised access to computer systems and wireless networks is a serious criminal offence in most countries. In Belgium, where this guide was written, the Computer Crime Act and the Cybersecurity Act both carry real penalties — including imprisonment — for unauthorised interception, interference, and access. Similar laws exist across the EU under the NIS2 Directive framework, and globally under laws like the US Computer Fraud and Abuse Act, the UK Computer Misuse Act, and countless equivalents.

A Marauder device used irresponsibly is not a grey area — it is evidence. The logs you generate, the captures you make, and the signals you transmit can all be traced. The risk is real and the consequences are serious.

The Principled Case

Beyond the legal risk, there is a simpler reason to stay white hat: the people on the other side of those networks are real. The devices you see in a Station Scan belong to someone. The probe requests you capture came from someone's phone. The networks you see in an AP Scan are someone's home, someone's business, someone's livelihood.

The security community at its best is built on trust — trust that knowledge will be used responsibly, that vulnerabilities will be disclosed to the people who can fix them, and that the tools we build and share will make the world more secure rather than less. Every person who uses a tool like Marauder responsibly reinforces that trust. Every person who does not erodes it for everyone.

The Community Case

Open-source security tools exist because developers trust the community to use them well. When tools get misused publicly — when someone uses a Marauder to disrupt a café's Wi-Fi, or

deploys an evil portal against unsuspecting strangers — it invites regulatory scrutiny, platform bans, and the kind of attention that makes it harder for everyone to learn, experiment, and build.

Staying white hat is not just about protecting yourself. It is about protecting the community, the tools, and the open culture of learning that makes projects like ESP32 Marauder possible in the first place.

A Simple Test

When you are about to use any Marauder feature and you are not sure whether it is appropriate, ask yourself three questions:

- Do I own this network or device, or do I have explicit written permission from the person who does?
- Would I be comfortable explaining exactly what I am doing and why to a law enforcement officer or a judge?
- Could this action cause harm — even unintentionally — to someone who did not consent to it?

If the answer to the first two is yes and the third is no, you are on solid ground. If you hesitate on any of them, stop and reconsider.

The goal is to be the kind of person who makes the internet safer — not the kind who makes it more dangerous. Everything in this guide is written with that goal in mind.

The Feature Tier System — Explained

Marauder comes with a lot of functions. Some you will use every single session. Others you may never touch. Some are safe to run anywhere with zero preparation. Others carry real legal risk and require careful thought before activating. Rather than presenting everything as a flat list and leaving you to figure out what matters, this guide organises Marauder's features into a tier system.

The tiers are not about which features are technically impressive or which ones sound the most exciting. They are about practical, day-to-day value for a responsible user — weighted against accessibility, legal risk, and how much prior knowledge is needed to use each feature meaningfully.

How the Tiers Are Assigned

Each feature is evaluated against four criteria:

- Practical value: How often will a typical hobbyist or beginner actually use this? Does it produce results that are genuinely useful or educational in everyday situations?
- Accessibility: Can a beginner run this on day one with no prior experience, or does it require significant setup, companion tools, or background knowledge to get anything useful out of it?

- Legal risk: Is this a passive observation function that carries minimal risk, or does it transmit frames, interfere with devices, or capture material that could expose the user to legal liability if misused?
- Dependency: Does this feature stand alone, or does it only make sense as part of a larger workflow involving other tools, specific hardware, or advanced knowledge?

The Four Tiers

S-Tier — Most Practical / High-Value Features

S-tier features are the heart of Marauder. They work reliably out of the box, produce immediately useful and educational results, carry low legal risk when used responsibly, and form the foundation of nearly every workflow. If you are new to Marauder, S-tier is where you spend most of your time. AP Scan, Station Scan, Probe Sniff, Wardriving, PMKID Sniff, and Save PCAP all sit here — not because they are the flashiest features, but because they are the most consistently valuable ones.

A-Tier — Strong Utility / Situationally Excellent

A-tier features are genuinely powerful and reward the time investment generously — but they shine brightest in specific contexts rather than every session. They may require a bit more background understanding to get full value from, or they may be most useful to users who have already built comfort with the S-tier workflow. Beacon Sniff, Deauth Sniff, and Bluetooth scanning sit here: highly capable, lower risk, but slightly more specialised than the S-tier essentials.

B-Tier — Useful but Specialised / Higher Risk

B-tier features are where the legal risk profile increases noticeably. These functions actively transmit into the wireless environment — they are not passive observers. Evil Portal creates a rogue access point. Beacon Spam floods the airwaves with fake networks. Both have legitimate educational uses in controlled, authorised settings, but both can cause real harm and real legal consequences if deployed carelessly or maliciously. B-tier features reward users who understand them fully and use them exclusively in appropriate contexts. They are not features to run experimentally in public to see what happens.

C-Tier — Demo / Novelty / Experimental

C-tier features have limited practical value in a real-world security or analysis context. They are curiosities, entertainment functions, or experimental tools that demonstrate a concept or produce an amusing effect without contributing much to a meaningful workflow. Probe Request Flood, BLE Spam, and the Rick Roll beacon live here. They are not useless — understanding why they work is genuinely educational — but they are not tools you reach for when doing serious work.

A Note on Tier Placement

The tiers reflect the perspective of a responsible hobbyist or beginner building genuine skills. A professional penetration tester working on an authorised engagement might place Evil Portal in S-tier because it is central to their workflow. A pure wardriving enthusiast might barely touch anything below A-tier. The tiers are a guide for navigating the feature set sensibly — not a rigid hierarchy.

As your skills and confidence grow, your personal tier list will shift. Features that felt intimidating or inaccessible at first become routine. The goal of this system is simply to give you a clear starting point: begin with S-tier, build your understanding, and work your way outward from there at your own pace.

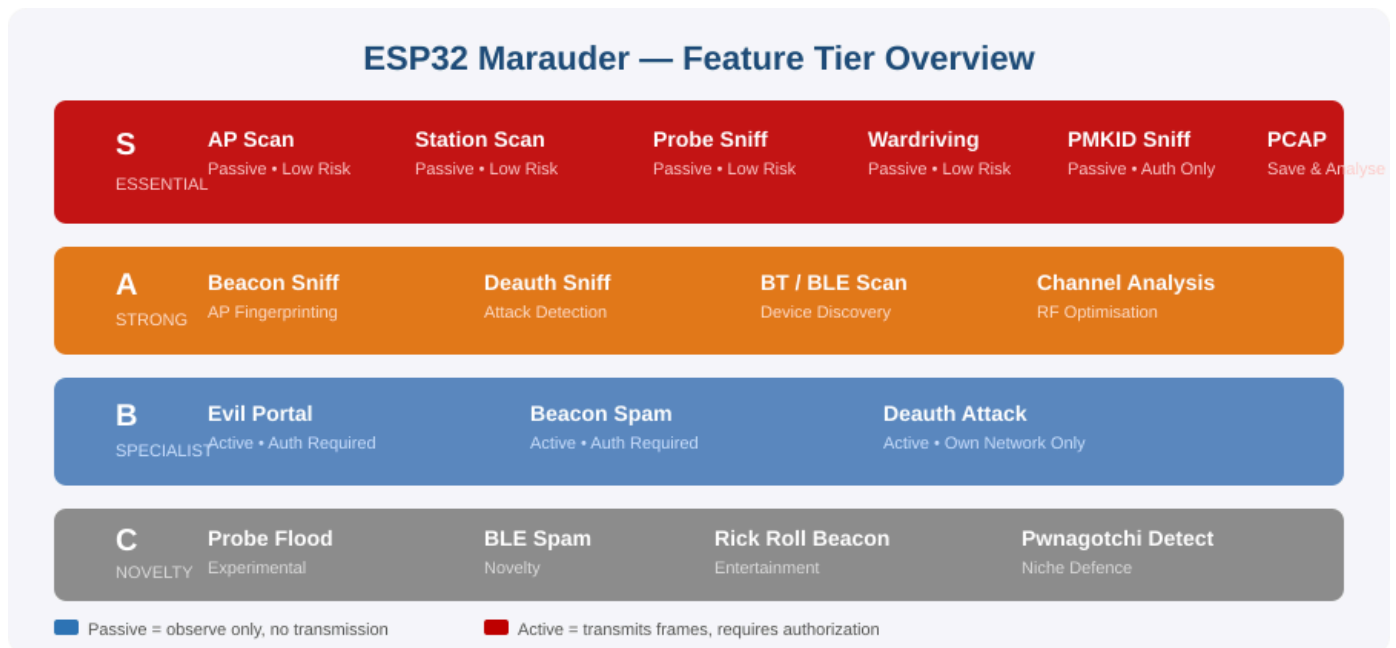


Figure 1 — Feature Tier Overview: S through C, colour-coded by risk and utility

S-Tier: Most Practical / High-Value Features

These features deliver the most consistent day-to-day value. If you are new to Marauder, this is where to start. They are low-risk, broadly useful, and form the foundation of nearly every workflow.

[S-TIER] AP Scan

What it does: Detects nearby Wi-Fi access points and lists each network's SSID (name), BSSID (hardware address), RSSI (signal strength), channel, and encryption type.

Why It Matters for Beginners

AP Scan is the starting point for almost everything you do with Marauder. Before you can analyze, log, or investigate any wireless environment, you first need to know what networks exist around you. This function gives you exactly that — a live snapshot of the wireless landscape.

Every access point (AP) in range will appear in the list. You will immediately learn which networks are nearby, how strong their signals are, what channel they operate on, and whether they use encryption. This is the wireless equivalent of opening your eyes.

Reading the Results

- **SSID:** The human-readable name of the network (e.g., 'HomeNetwork_5G'). Hidden networks may appear with a blank SSID.
- **BSSID:** The unique hardware MAC address of the access point. Useful for distinguishing two networks with the same name.
- **RSSI:** Signal strength in dBm. Values closer to 0 are stronger (e.g., -40 dBm is very strong; -85 dBm is very weak).
- **Channel:** The frequency channel the AP broadcasts on. Common channels are 1, 6, and 11 in the 2.4 GHz band.
- **Encryption:** The security protocol in use — WPA2, WPA3, WEP, or Open (none).

Real-World Uses

- **RF surveying:** See how many networks exist in an area and how they are distributed across channels.
- **Wardriving preparation:** Run AP Scan before logging to confirm GPS and scanning are both active.
- **Home network troubleshooting:** Identify which channels nearby networks are using to reduce interference on your own router.
- **Security awareness:** Check whether your own network is broadcasting the encryption type you expect.

 **Tip:** Run an AP Scan first whenever you arrive somewhere new. It takes only seconds and gives you the context needed for every follow-up function.

Skill Level: Beginner

Legal Risk: None — passive observation only

[S-TIER] Station Scan

What it does: Detects client devices (phones, laptops, smart TVs, IoT gadgets) that are actively associated with nearby access points, and maps which client belongs to which network.

Why It Matters for Beginners

AP Scan tells you about the networks. Station Scan tells you about the devices connected to those networks. Together they paint a much fuller picture of the wireless environment.

When a phone connects to a router, it becomes a 'station.' Marauder can detect these stations and show you which AP each one is connected to, along with the device's MAC address and signal strength. This is especially useful for understanding how busy a network is or for inventorying devices on your own home network.

What You Will See

- Client MAC addresses: Unique identifiers for each connected device.
- Associated AP (BSSID): Which access point each client is connected to.
- RSSI: How strong the signal is between the client and the AP.

Real-World Uses

- Home network audit: Count how many devices are actually connected to your router and identify anything unexpected.
- Device mapping: Understand the relationship between clients and APs in a multi-router environment.
- Education: Visually demonstrate how wireless client-AP associations work in a classroom or lab setting.



Tip: Run Station Scan immediately after AP Scan for a complete picture. The AP list gives you the anchors; the station list fills in the devices around them.

Skill Level: Beginner to Intermediate

Legal Risk: None on passive observation — do not use data for unauthorized purposes

[S-TIER] Probe Sniff

What it does: Captures probe request frames — small wireless signals that client devices broadcast when searching for Wi-Fi networks they have connected to previously.

Understanding Probe Requests

Every time your phone or laptop hunts for a familiar network, it sends out a probe request containing the SSID (name) of the network it is looking for. This happens automatically in the background, even when you are not actively using the device.

This is a fascinating privacy consideration: your device is essentially announcing the names of every Wi-Fi network you have ever connected to, broadcasting them into the air for anyone with the right tools to hear. Marauder's Probe Sniff captures these broadcasts passively.

Why This Is Valuable to Learn

- Privacy demonstrations: Show friends or colleagues how their device history leaks into the wireless environment without their awareness.
- Roaming behavior: Understand how devices search for preferred networks when moving between locations.
- Security awareness: Illustrate why connecting to public Wi-Fi hotspots can expose your device's network history.
- Client analysis: In an authorized lab environment, probe data can help map which devices have previously connected to which networks.

⚠ Capturing probe requests from others' devices without authorization may be regulated in your jurisdiction. Use this feature for education, your own devices, or explicitly authorized testing only.

💡 Tip: Try this at home with your own phone. Turn Wi-Fi off, then back on, and watch Marauder capture your device broadcasting the names of your saved networks. It is an eye-opening privacy demonstration.

Skill Level: Intermediate

Legal Risk: Low to Moderate — know your local laws around passive wireless capture

[S-TIER] PMKID Sniff

What it does: Captures PMKID (Pairwise Master Key Identifier) authentication material when it is exposed during the initial connection handshake between a client and an access point.

What Is a PMKID?

When a client device connects to a WPA2-protected network, the access point and client exchange cryptographic material as part of establishing a secure session. In some configurations, the AP will include a PMKID in the first message of this handshake — and Marauder can capture it without needing to wait for a full four-way handshake to complete.

The PMKID is derived from the network's password (pre-shared key), meaning that in an authorized security assessment, a captured PMKID can be analyzed offline to evaluate how strong or guessable the network password is.

Practical Uses in Authorized Testing

- Auditing your own network: Capture your own PMKID and use offline tools to verify how resistant your Wi-Fi password is to dictionary or brute-force guessing.
- Security assessments: Professionals authorized to test a network can use PMKID capture as part of a broader credential-strength audit.
- Lab education: Demonstrate WPA2 handshake mechanics in a controlled environment.

⚠️ **Capturing PMKIDs from networks you do not own or are not authorized to test is illegal in most jurisdictions. This feature is strictly for authorized use. The legal risk is high if misused.**

💡 Tip: For beginners: think of PMKID capture as a way to 'take a snapshot' of the lock on a door. Whether you can actually pick the lock depends on offline analysis — but even taking the snapshot on someone else's door without permission is trespassing.

Skill Level: Intermediate to Advanced

Legal Risk: High if used without authorization

[S-TIER] Wi-Fi Wardriving

What it does: Logs discovered Wi-Fi networks along with GPS coordinates, creating a geolocated dataset of wireless environments as you travel.

What Is Wardriving?

Wardriving is the practice of moving through physical space — driving, cycling, walking — while your device scans for Wi-Fi networks and records their locations using GPS. The result is a map of wireless networks tied to real-world geography.

Despite the name, wardriving is a long-standing and largely passive hobby with a thriving community. Modern wardrivers use it to contribute to global wireless mapping projects, track RF coverage in their area, and participate in friendly community competitions.

How Marauder Does It

- Attach a compatible GPS module to your Marauder device.
- Verify a GPS fix using the GPS Monitor function (look for satellite count and coordinate lock).
- Start Wi-Fi Wardrive Logging — Marauder will automatically associate each detected AP with your current GPS coordinates.
- Optionally, also enable Bluetooth Wardrive Logging to capture BLE devices simultaneously.
- When finished, export your log files from the SD Card Browser.

The Wardriving Community

Many Marauder users upload their logs to WiGLE (Wireless Geographic Logging Engine), a global community platform where wardrivers contribute to a continuously growing wireless map of the world. WiGLE lets you track your personal statistics, compare coverage with others, join teams, and participate in leaderboard competitions.

 **Tip:** Always verify your GPS fix before starting a wardriving session. A solid fix typically requires a few minutes outdoors with clear sky visibility. Without a fix, your logs will have no location data.

 **Review WiGLE's terms, privacy policy, and submission guidelines before uploading any collected data.**

Skill Level: Beginner

Legal Risk: Low — passive observation; check local regulations on GPS logging in public spaces

[S-TIER] Save PCAP

What it does: Saves raw captured wireless frames to the SD card in PCAP format — a standardized file type readable by professional analysis tools like Wireshark.

What Is a PCAP File?

PCAP stands for Packet Capture. It is the universal standard format for storing raw network traffic, used by security professionals, network engineers, and researchers worldwide. When Marauder saves a PCAP, it is recording wireless frames exactly as they were observed in the air — a precise snapshot of radio activity.

PCAP files can be opened in Wireshark, one of the most widely used network analysis tools in the world (and free to download). Wireshark lets you inspect each captured frame individually, filter by type, follow conversations, and decode protocol details.

Why Beginners Should Learn This

- Education: Opening a PCAP from your own home network in Wireshark is one of the best ways to understand how Wi-Fi actually works at the protocol level.

- Offline analysis: Capture now, analyze later — you do not need to interpret everything in real time on the device's small screen.
- Troubleshooting: Save captures of unusual network behavior and analyze them later in detail.
- Authorized security testing: In a homelab or authorized assessment, PCAPs containing authentication material can be used to audit credential strength.

Optional Advanced Analysis (Authorized Only)

In authorized security testing workflows, captured authentication data may be evaluated using password-auditing tools such as Hashcat to assess Wi-Fi credential strength. This is strictly for defensive security testing in environments you own or are explicitly permitted to test.

⚠ Never capture or analyze authentication material from networks you do not own. This applies even if the network is unencrypted or the signal reaches your location.

💡 Tip: Download Wireshark for free and open one of your own home network PCAPs. Filter by 'eapol' to see WPA2 handshake frames. It is the fastest way to move from beginner to intermediate understanding of wireless security.

Skill Level: Intermediate

Legal Risk: Low for own networks; High if used to capture or analyze others' traffic

A-Tier: Strong Utility / Situationally Excellent

These features are highly useful in the right context. They may require a bit more understanding to get full value from, but they reward the time investment generously.

Beacon Sniff

What it does: Passively captures beacon frames — the regular broadcast announcements that access points transmit roughly every 100 milliseconds.

Understanding Beacon Frames

Beacon frames are the heartbeat of any Wi-Fi network. Every access point continuously broadcasts them to announce its presence, advertise its capabilities, and let nearby devices know it is available. A single active AP will transmit around 10 beacon frames per second.

Each beacon contains a rich set of information: the SSID, supported data rates, security capabilities, channel information, and vendor-specific data. Capturing beacons allows you to fingerprint access points in detail — identifying manufacturer, firmware generation, and configuration specifics that simple scanning does not reveal.

Practical Uses

- AP fingerprinting: Identify router manufacturers and sometimes firmware versions from beacon content.

- Hidden SSID detection: While hidden networks suppress their SSID in beacons, they still beacon regularly and can be detected by their BSSID.
- Channel monitoring: Observe which channels are congested by counting beacon density.
- Vendor analysis: Decode vendor-specific information elements embedded in beacons.

Skill Level: Intermediate

Legal Risk: Low — passive capture of publicly broadcast frames

Deauth Sniff

What it does: Listens for deauthentication frames in the wireless environment and alerts you when they are detected.

Why Deauth Detection Matters

Deauthentication frames are a normal part of the Wi-Fi protocol — they are how an access point or client officially ends a connection. However, they are also famously spoofable: because the original 802.11 standard did not require these frames to be authenticated, an attacker can send forged deauth frames to forcibly disconnect devices from a network.

Deauth Sniff turns Marauder into an early warning system. If someone nearby is running a deauthentication attack — for example to force a WPA2 handshake capture — Deauth Sniff will detect the flood of deauth frames and alert you.

- Network security monitoring: Identify potential wireless disruption attacks on your own network.
- Education: Demonstrate how deauth attacks work by observing the frame signature passively.
- Event security: Use at a conference or venue (with authorization) to detect wireless interference attempts.

Skill Level: Intermediate

Legal Risk: None for passive detection

Bluetooth / BLE Scan

What it does: Discovers and lists Bluetooth Classic and Bluetooth Low Energy (BLE) devices broadcasting in range.

Most consumer electronics constantly broadcast Bluetooth or BLE advertisements to announce their presence. Headphones, smartwatches, fitness trackers, smart home sensors, medical devices, and proximity beacons all appear in a BLE scan.

Unlike Wi-Fi scanning, BLE scanning does not require pairing or connection — devices advertise openly by design. Marauder captures these advertisements and displays device names, MAC addresses, and signal strength.

- BLE device inventory: See exactly how many Bluetooth devices are in a given space.
- Privacy demonstrations: Show how many devices around you are broadcasting identifying information.

- IoT research: Identify BLE sensors and beacons in a smart home or office environment.
- AirTag / tracker detection: Find broadcasting tracking devices in your vicinity (see AirTag Monitor).

Skill Level: Beginner to Intermediate

Legal Risk: Low — passive scanning of public BLE advertisements

B-Tier: Useful but Specialized / Niche

These features are powerful in the right hands and right context, but come with higher legal risk and a steeper learning curve. Beginners should understand the legal boundaries thoroughly before experimenting.

[B-TIER] Evil Portal

What it does: Hosts a fake captive portal web page via a rogue access point, simulating the login screens commonly seen at hotels, airports, and coffee shops.

What Is a Captive Portal?

When you connect to hotel or airport Wi-Fi and get redirected to a login or terms-of-service page before accessing the internet, that is a captive portal. Evil Portal creates a similar experience — Marauder broadcasts an open AP, and any device that connects to it is redirected to a custom web page served by Marauder.

Legitimate Uses

- Security awareness training: Demonstrate phishing via fake portals in a controlled, authorized educational setting.
- Captive portal development: Test custom portal designs and user flows for legitimate hotspot deployments.
- Homelab experimentation: Learn how captive portal infrastructure works in a closed lab network.

⚠ Using Evil Portal against real users without their explicit consent is illegal in virtually every jurisdiction. This includes operating it in a public space even if you do not actively solicit connections. The legal risk is extremely high.

💡 Tip: For a safe and legal demonstration, connect your own phone to the rogue AP you created and walk a trusted friend through what a phishing portal looks like. Never deploy this on a live network or in public.

Skill Level: Intermediate to Advanced

Legal Risk: Extremely High if used against real users without consent

[B-TIER] Beacon Spam (Random & List)

What it does: Broadcasts large numbers of fake Wi-Fi network names (SSIDs), causing nearby devices to 'see' dozens or hundreds of networks that do not actually exist.

Two Modes

- Beacon Spam Random: Generates random, nonsensical SSIDs. Mostly useful for demonstrating how easily the Wi-Fi network list can be flooded.
- Beacon Spam List: Broadcasts a custom list of SSIDs you define. Can be used for educational demonstrations or testing how Wi-Fi clients behave when presented with many network options.
- Rick Roll Beacon: A preconfigured novelty variant that spams song-title SSIDs. Entertainment value only.

Educational Value

Beacon spam is a useful demonstration of a well-known Wi-Fi weakness: because the standard does not authenticate beacon frames, any device can broadcast any SSID. Seeing a phone's network list suddenly fill with hundreds of fake networks is an effective way to illustrate this concept to a non-technical audience.

⚠ Beacon spam can disrupt nearby Wi-Fi clients and management tools, and transmitting fake SSIDs may violate radio regulations in your jurisdiction. Even with benign intent, running this in a public or shared space can cause problems for others. Legal risk ranges from Moderate to High.

💡 Tip: If you want to experiment with beacon spam safely, do it in a controlled indoor space (your home) and keep the session brief. Never run it in public, at work, or anywhere it might affect others.

Skill Level: Beginner (to run) / Intermediate (to understand implications)

Legal Risk: Moderate to High depending on jurisdiction and context

C-Tier: Demo / Novelty / Experimental

These features are mostly for curiosity, entertainment, or experimental development. They have limited practical value in a security or analysis context but can be educational in controlled settings.

Probe Request Flood

Floods the wireless environment with probe request frames. Primarily experimental — useful for observing how APs respond to high volumes of probe traffic, but has limited real-world application outside of controlled lab testing.

BLE Spam / BT Spam

Broadcasts high volumes of BLE advertisement packets. Mostly a novelty or development testing tool. On iOS devices, certain BLE advertisement types can trigger pop-up notifications (a well-known 'nuisance' effect). Using this in public is inconsiderate at best and potentially illegal at worst.

Detect Pwnagotchi / Card Skimmers

Looks for the characteristic wireless signatures of Pwnagotchi devices (automated WPA handshake harvesters) and certain types of card skimmer Bluetooth transmitters. A highly niche but genuinely useful defensive function in specific contexts — for example, checking an ATM vicinity for skimmer Bluetooth signatures.

Supporting & Utility Features

GPS Monitor

Displays real-time GPS coordinates, fix status, satellite count, speed, and altitude. Essential for wardriving — always verify your GPS fix here before starting a logging session.

SD Card Browser

Manage files stored on the SD card: view, rename, or delete PCAP captures, wardriving logs, Evil Portal HTML files, and other saved data. Essential for keeping your storage organized across sessions.

CLI / Serial Console

Allows you to control Marauder via text commands over a UART serial connection. Advanced users can script and automate Marauder functions from a connected computer, making it possible to build custom workflows, logging pipelines, and integration with other tools.

Recommended Workflows for Beginners

Workflow 1: Basic Recon / RF Surveying

- Step 1 — Run AP Scan to discover nearby networks.
- Step 2 — Run Station Scan to map connected clients.
- Step 3 — Run Beacon Sniff for AP fingerprinting.
- Step 4 — Use Channel Analyzer to assess channel congestion.

Workflow 2: Authorized Security Auditing

- Step 1 — AP Scan to identify target network.

- Step 2 — Station Scan to map active clients.
- Step 3 — Probe Sniff to capture client network history.
- Step 4 — PMKID Sniff to capture authentication material.
- Step 5 — Save PCAP for offline analysis in Wireshark.

Workflow 3: Wardriving

- Step 1 — Verify GPS fix via GPS Monitor.
- Step 2 — Start Wi-Fi Wardrive Logging.
- Step 3 — Optionally start Bluetooth Wardrive Logging.
- Step 4 — Travel route while Marauder logs automatically.
- Step 5 — Export logs from SD Card Browser.
- Step 6 — Optionally upload to WiGLE or similar platform.

Wireless Theory Primer

Understanding what ESP32 Marauder is actually doing requires a basic grasp of how wireless networking works at the protocol level. This section explains the core concepts in plain language — no prior networking knowledge assumed.

1. What Wireless Communication Actually Is

Wireless networks transmit data as radio waves through the air, primarily in the 2.4 GHz and 5 GHz frequency bands for Wi-Fi, and 2.4 GHz for Bluetooth and BLE. Rather than sending one continuous stream, data is broken into small, discrete chunks called frames. These frames are broadcast into the air, received by nearby antennas, decoded, and interpreted.

ESP32 Marauder places the ESP32's wireless radio into a special mode that lets it observe these frames directly — before they are filtered and abstracted by a normal operating system. This is why Marauder can see traffic that your laptop's Wi-Fi settings screen never shows you.

2. Wi-Fi Fundamentals (IEEE 802.11)

Wi-Fi networks are built around two types of participants: access points (APs) and client devices (stations).

Access Points

An AP is typically a router or base station. It continuously broadcasts beacon frames roughly every 100 milliseconds, advertising its SSID (network name), supported speeds, security protocol, and capabilities. Beacons are what allow your phone to 'see' available networks.

Client Devices (Stations)

Clients are phones, laptops, smart TVs, IoT sensors — anything that connects to a network. When a client wants to find a known network, it sends probe request frames containing the

SSIDs of networks it has previously connected to. When it connects, it completes an authentication and association handshake with the AP.

Key Frame Types

- Beacon Frames: Sent by APs ~10 times per second. Announce network presence and capabilities.
- Probe Requests: Sent by clients searching for remembered networks. Reveal device history.
- Probe Responses: AP replies to a probe request, confirming network availability.
- Authentication Frames: Initial handshake steps when a client attempts to join a network.
- Association Frames: Finalize the client-AP connection after authentication.
- Deauthentication Frames: Formally end a client-AP session. Can be spoofed — a known protocol weakness.
- Data Frames: The actual user traffic once a connection is established.

3. RSSI and Signal Strength

RSSI (Received Signal Strength Indicator) measures how strong a wireless signal is at the receiving antenna. It is expressed in dBm — always a negative number. The closer to zero, the stronger the signal.

- -30 dBm: Extremely strong. Device is very close to the AP.
- -60 dBm: Good. Normal home or office range.
- -75 dBm: Fair. Usable but degraded performance.
- -85 dBm or lower: Weak. Edge of detection range.

 **Tip:** Use RSSI during wardriving to gauge how close you are to a detected network. A very strong signal from an unknown network in an unexpected location can indicate a rogue or unusually positioned AP.

4. Channels and Frequency Bands

Wi-Fi networks do not all operate on the same frequency — they are divided into numbered channels within two main bands.

2.4 GHz Band

Channels 1 through 13 (exact availability varies by country). Offers longer range and better wall penetration, but is more congested — all Bluetooth devices, microwave ovens, and many IoT devices also share this band. The non-overlapping channels in most regions are 1, 6, and 11.

5 GHz Band

Many more channels available, faster speeds, shorter range, and significantly less interference. Most modern routers are dual-band (2.4 + 5 GHz). Marauder can scan channels in both bands.

 **Tip:** If your home network feels slow or unreliable, run a Channel Analyzer scan. If channels 1, 6, or 11 are congested with neighboring networks, changing your router's channel may noticeably improve performance.

5. BLE (Bluetooth Low Energy) Basics

BLE is a low-power variant of Bluetooth designed for devices that need to run for months or years on a small battery: fitness trackers, smart home sensors, proximity beacons, medical devices, and asset trackers like AirTags.

BLE devices communicate primarily through advertisements — small packets broadcast periodically by the device without requiring any pairing or connection. These advertisements contain the device name, service identifiers, and manufacturer-specific data. Because they are broadcast openly, any BLE scanner (including Marauder) can receive and log them.

- Advertisement interval: Typically between 100ms and several seconds depending on the device's power settings.
- Three advertising channels: BLE uses channels 37, 38, and 39 specifically for advertisement broadcasts.
- No pairing required: Advertisements are visible to all BLE scanners by design.

6. Why Packet Capture Matters

When Marauder saves wireless traffic to a PCAP file, it is creating a precise, frame-by-frame record of everything the radio observed. Unlike the abstracted view your operating system gives you ('connected to HomeNetwork, 74 Mbps'), a PCAP shows you the actual bytes exchanged at the protocol level.

This granularity is what makes PCAPs so valuable for education, troubleshooting, and authorized security testing. In Wireshark, you can reconstruct exactly what happened during a WPA2 handshake, observe probe request sequences, or study beacon frame structure — all from your own home network, for free.

7. Passive vs Active Monitoring

Passive Monitoring

Listening-only mode. Marauder receives frames but transmits nothing. AP Scan, Station Scan, Probe Sniff, Beacon Sniff, BLE Scan, and Wardriving are all passive. The device is invisible to the networks and devices it observes.

Active Interaction

Marauder transmits frames into the environment. Evil Portal, Beacon Spam, Deauth attacks, and Probe Flood are all active. Active transmission is what creates legal risk — you are no longer just observing; you are participating in the wireless environment.

⚠ Understanding the passive/active distinction is critical. Passive functions are generally lower risk; active functions can cause interference or harm and require explicit authorization.

Deep Dive: Decoding the WPA2 Four-Way Handshake

Understanding what Marauder actually captures during a PMKID sniff or a triggered WPA2 handshake is the difference between being a button-pusher and being an analyst. This section explains the cryptographic exchange at the heart of WPA2 security — clearly, without unnecessary jargon.

Why the Handshake Matters

When a client device connects to a WPA2-protected network, neither side transmits the actual password over the air. Instead, they run through a four-step cryptographic exchange called the EAPOL four-way handshake that proves both sides know the password without ever revealing it. This is elegant security design — but the handshake itself contains enough mathematical material to test the password offline, which is why capturing it is valuable in an authorized security audit.

The Four Steps — What Actually Happens

Step 1 — ANonce (AP to Client)

The access point sends a random number called the ANonce (Authenticator Nonce) to the connecting client. This is the AP's contribution to the shared randomness that will be used to derive session keys. At this point, no keys have been verified and no authentication has occurred.

Step 2 — SNonce + MIC (Client to AP)

The client generates its own random number, the SNonce (Supplicant Nonce), and uses both nonces together with the pre-shared key (the Wi-Fi password) and both MAC addresses to derive a Pairwise Transient Key (PTK). It then sends the SNonce to the AP along with a Message Integrity Code (MIC) — a cryptographic signature that proves the client knows the password, without transmitting the password itself.

This is the critical step for security analysis. The MIC is computed from the PTK, which is derived from the password. An analyst with the captured handshake can compute PTK candidates from a wordlist and check whether the resulting MIC matches the captured one — this is exactly what Hashcat does in an authorized credential audit.

Step 3 — GTK (AP to Client)

The AP verifies the client's MIC, confirming the client knows the password. It then sends the Group Transient Key (GTK) — the key used for broadcast and multicast traffic on the network — encrypted with the newly established PTK. This step completes the mutual authentication.

Step 4 — ACK (Client to AP)

The client sends a final acknowledgement confirming receipt of the GTK. Both sides now share the same session keys and encrypted data traffic begins. The four-way handshake is complete.

The Golden Handshake

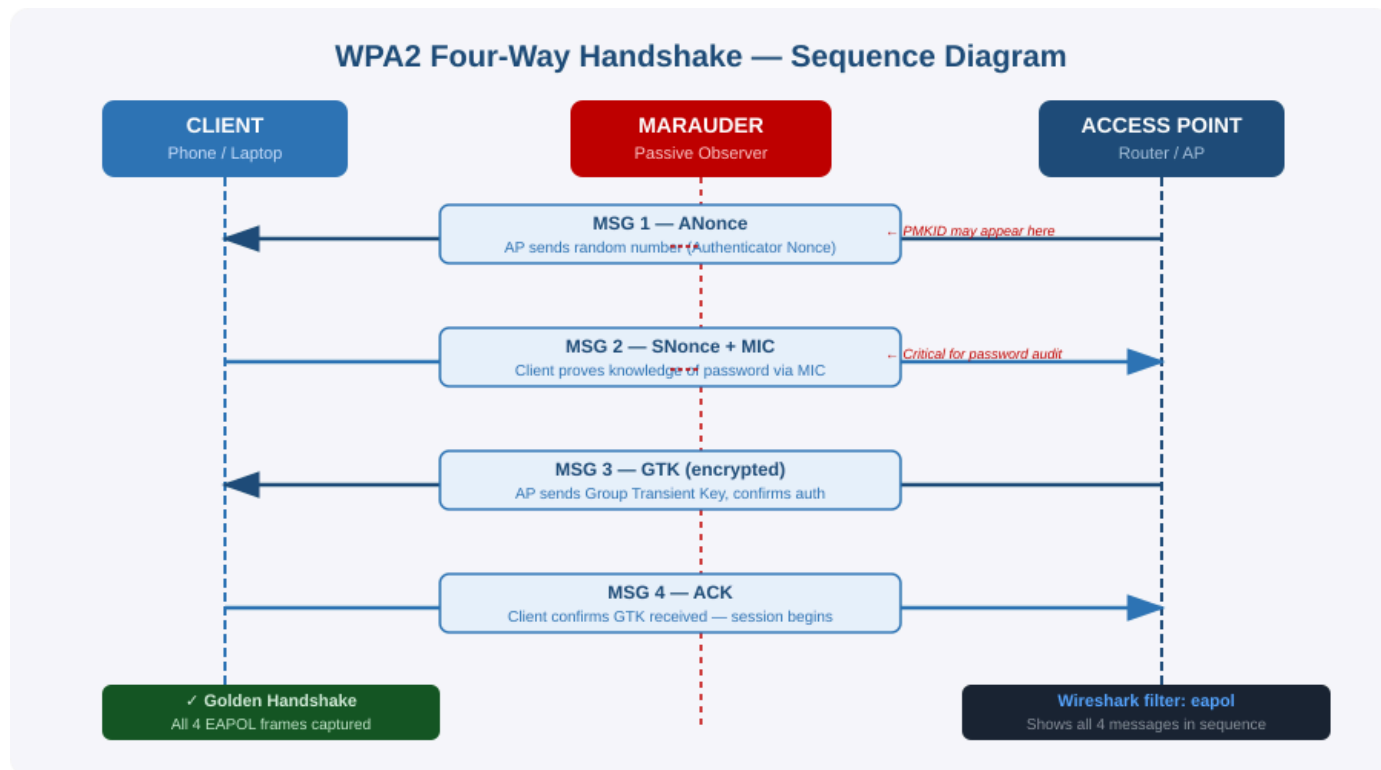


Figure 3 — WPA2 Four-Way Handshake sequence: what Marauder captures and why it matters

In Wireshark, filter your PCAP by 'eapol' to isolate handshake frames. A complete, usable handshake consists of all four EAPOL messages. When all four are present this is called a Golden Handshake — it contains everything needed for an offline password audit using tools like Hashcat in an authorized assessment context.

If you only see two or three EAPOL frames, the handshake was incomplete — possibly because the client disconnected mid-exchange or Marauder missed one of the frames due to channel timing. Trigger a fresh reconnection and try again.

 **Tip:** In Wireshark, click on any EAPOL frame and expand 'IEEE 802.1X Authentication' in the packet detail panel. You will see the Key Nonce field (the ANonce or SNonce depending on the frame), the Key MIC field, and whether the Install, Ack, and Secure flags are set. These flags tell you exactly which of the four steps each frame represents.

PMKID — The Shortcut

The PMKID (Pairwise Master Key Identifier) is a value included by many access points in the first EAPOL frame. It is derived directly from the PMK (Pairwise Master Key), which is itself derived from the Wi-Fi password. This means a PMKID capture alone — without needing to wait for a client to connect — can be sufficient for an authorized offline password audit.

Marauder's PMKID Sniff function specifically looks for this value in the first EAPOL message from access points. It does not require any client to be present or connecting — the AP exposes the PMKID when it initiates the handshake process, even before client authentication begins.

Not all access points expose a PMKID. Modern WPA3 networks do not use PMKID at all. But for WPA2 networks using the common PSK (pre-shared key) mode, PMKID capture is often faster and more reliable than waiting for a full four-way handshake.

⚠ Both PMKID capture and four-way handshake capture must only be performed on networks you own or are explicitly authorized to test. The data captured is sensitive — treat it accordingly and delete it after your authorized assessment is complete.

From Capture to Analysis — The Authorized Workflow

- Step 1 — Capture a PMKID or full four-way handshake from your own network using Marauder with Save PCAP enabled.
- Step 2 — Transfer the PCAP to your computer.
- Step 3 — Verify in Wireshark (filter: eapol) that you have a complete handshake or a valid PMKID.
- Step 4 — Convert the PCAP to Hashcat format using hcxtools: `hcxpcapngtool -o hash.hc22000 capture.pcap`
- Step 5 — Run Hashcat against the hash with a wordlist: `hashcat -m 22000 hash.hc22000 wordlist.txt`
- Step 6 — If your password appears in the wordlist, change it immediately to a long, random passphrase.

💡 Tip: The RockYou wordlist (available freely online) contains over 14 million commonly used passwords. If your Wi-Fi password appears in it, it is genuinely dangerous. A password that survives a RockYou test is significantly more secure — but a truly strong password is one that would not appear in any wordlist at all.

ESP32 Marauder vs Flipper Zero

A common comparison in the hardware community is between ESP32 Marauder and the Flipper Zero. Both are portable, hackable, multi-tool devices with enthusiastic communities — but they are designed with fundamentally different goals.

Core Design Philosophy

ESP32 Marauder

Marauder is a specialist. Its entire focus is Wi-Fi and Bluetooth (BLE) analysis. Every feature — scanning, wardriving, packet capture, PMKID sniffing — is built around deep visibility into wireless networking behavior. It excels at what it does precisely because it does not try to do everything.

Flipper Zero

Flipper Zero is a generalist. It is designed to interact with a wide range of short-range radio technologies — RFID, NFC, Sub-GHz signals, infrared, GPIO, and more. Breadth is its strength; it can touch many different protocols that Marauder cannot. However, its Wi-Fi capabilities depend on an add-on module (which itself often runs Marauder firmware), and it does not match Marauder's depth in wireless network analysis.

Where Each Excels

Use Marauder When You Need

- Detailed Wi-Fi scanning, network discovery, and client mapping
- Probe request and beacon analysis
- GPS-based Wi-Fi and BLE wardriving with structured logs
- PCAP capture for offline Wireshark analysis
- PMKID capture for authorized credential auditing
- Channel utilization analysis and RF surveying

Use Flipper Zero When You Need

- RFID and NFC card reading, emulation, and cloning (on authorized cards)
- Sub-GHz signal capture and replay experimentation
- Infrared remote control functionality
- GPIO and hardware prototyping and signal analysis
- A broad, single-device toolkit across many unrelated protocols

Practical Decision Guide for Beginners

If your primary interest is wireless network security, wardriving, packet capture, and understanding how Wi-Fi and Bluetooth work — choose Marauder. It is more focused, cheaper to build yourself, and goes deeper in its domain.

If your interests are broader — you want to experiment with garage door remotes, NFC tags, IR blasters, and RFID cards in addition to Wi-Fi — Flipper Zero offers that range. Many enthusiasts eventually end up owning both.

 **Tip:** The Flipper Zero's Wi-Fi devboard often runs Marauder firmware. So if you already own a Flipper Zero, you may already have Marauder — just on a smaller screen with a different form factor.

Community Perspective

Neither device is universally 'better.' They are tools optimized for different jobs. The communities overlap significantly, and many enthusiasts use both together: Marauder for wireless network analysis and wardriving, Flipper Zero for multi-protocol signal interaction. Used in combination, they cover a much broader range of hardware security experimentation than either alone.

Final Assessment

ESP32 Marauder is best understood as a portable Wi-Fi reconnaissance and analysis platform that happens to also have Bluetooth/BLE capabilities, a wardriving mode, and (for advanced users) authorized security testing functions.

Its flashier features — beacon spam, deauth detection, evil portal — attract attention, but the platform's greatest long-term value lies in its recon, analysis, logging, and educational capabilities. For anyone curious about how wireless networks actually work, Marauder is one of the most accessible and hands-on learning tools available.

Marauder Is Best Used As

- A portable Wi-Fi reconnaissance and analysis platform
- A BLE and Wi-Fi wardriving tool with GPS logging
- A packet capture and RF survey device
- A wireless education and demonstration platform

⚠ Always use Marauder responsibly: only on networks and devices you own or are authorized to test. Know your local laws. Wireless security tools are powerful — treat them accordingly.

Community Note

Marauder firmware evolves frequently — features are added, refined, or renamed across releases and hardware targets. If sharing this guide publicly, keep it updated as the ecosystem grows. The community around this project is active, welcoming, and a great resource for questions and new firmware developments.

Compatible Hardware

One of Marauder's greatest strengths is how many different hardware configurations it supports. Whether you want a minimal \$5 dev board or a fully featured handheld with GPS, screen, and SD card, there is a build path for you.

Recommended ESP32 Development Boards

ESP32 Wroom / DevKit V1

The classic and most common starting point. Inexpensive, widely available, and well-supported by Marauder firmware. No built-in screen or SD card, but compatible with external add-ons. Best for beginners who want to experiment before committing to a full build.

- Pros: Cheap, widely available, easy to flash
- Cons: No screen or SD card built in — requires external components
- Best for: First-time builders and experimenters

ESP32-S2 / ESP32-S3

Newer ESP32 variants with USB native support and improved performance. Some Marauder builds target these specifically. Check the official firmware releases for S2/S3-specific builds before purchasing.

- Pros: USB native support, faster processor on S3
- Cons: Fewer community build guides than classic ESP32
- Best for: Intermediate builders wanting modern hardware

Flipper Zero WiFi Devboard

An ESP32-S2 module designed to plug directly into the Flipper Zero GPIO header. Runs Marauder firmware and gives Flipper Zero users Wi-Fi scanning and wardriving capabilities. Limited screen real estate but extremely convenient for existing Flipper owners.

- Pros: Plug-and-play with Flipper Zero, no soldering required
- Cons: Small display, dependent on Flipper Zero hardware
- Best for: Flipper Zero owners who want Marauder features

Official Prebuilt Marauder Devices

JustCallMeKoko and approved community vendors sell purpose-built Marauder hardware. These come pre-flashed, tested, and ready to use out of the box. They typically include an integrated screen, SD card slot, battery, and GPS support.

 **Tip:** If you are unsure about DIY assembly, buying an official device is the fastest route to a working, fully featured Marauder. Check JustCallMeKoko's [GitHub](#) and [community channels](#) for current availability.

Display Options

ILI9341 TFT Display (2.8" or 3.2")

The most common display used in DIY Marauder builds. Colorful, readable, and well-supported by the firmware. Connects via SPI.

ST7789 Display

Another popular option used in some community builds. Slightly different wiring, but fully supported. Check your specific firmware build for pinout details.

No Display

Headless operation is possible using the CLI/Serial Console over USB. Useful for automated or scripted workflows where a screen is unnecessary.

GPS Modules

NEO-6M / NEO-7M / NEO-8M (u-blox)

The most widely used GPS modules in Marauder builds. The NEO-6M is the cheapest and most common. All three work well for wardriving. Connect via UART (TX/RX pins).

- NEO-6M: Budget-friendly, widely available, 1Hz update rate
- NEO-7M: Slightly better sensitivity, same price range
- NEO-8M: Faster acquisition, better in challenging environments

💡 Tip: Allow 1-2 minutes outdoors for first GPS fix. Subsequent fixes are much faster once the module has almanac data cached.

SD Card Requirements

- Format: FAT32 (required — exFAT is not supported)
- Capacity: 32GB or smaller recommended for best compatibility
- Speed class: Class 10 or higher recommended for reliable PCAP writes
- Card type: Standard SD or MicroSD depending on your module/board

⚠️ **Always format your SD card as FAT32 before use. Cards formatted as exFAT or NTFS will not be recognized by Marauder.**

Battery Options

For portable builds, a single-cell LiPo (3.7V) battery connected through a TP4056-based charge controller is the community standard. Capacity depends on your enclosure size — 1000mAh gives roughly 3-5 hours of active use depending on screen brightness and GPS activity.

- 1000mAh LiPo: Compact, 3-5 hours use
- 2000mAh LiPo: Good balance of size and runtime
- 3000mAh+: Extended wardriving sessions, larger enclosure required

Advanced Antenna & Range Extension

The stock PCB trace antenna built into the ESP32-WROOM module is perfectly adequate for tabletop testing and casual wardriving. But once you start doing serious RF surveying, extended wardriving, or authorized remote assessments, understanding antenna theory and your options for extending range becomes genuinely valuable.

Onboard PCB Antenna vs External Antenna

ESP32-WROOM (Built-in PCB Antenna)

The standard ESP32-WROOM module uses a printed circuit board trace antenna integrated directly into the module. It is compact, requires no external hardware, and works well for most use cases within typical indoor ranges of 20-50 metres. The vast majority of Marauder builds use this variant.

ESP32-WROOM-U / ESP32-WROVER-U (U.FL External Connector)

The -U suffix variants of the ESP32-WROOM and ESP32-WROVER modules replace the PCB trace antenna with a U.FL connector — a small, surface-mount RF connector that accepts an external antenna via a U.FL to SMA adapter cable. This single hardware change unlocks a significant range improvement by allowing you to attach any standard 2.4 GHz antenna.

- Look for the module designation ending in -U when sourcing components if you want external antenna capability
- You will need a short U.FL to SMA pigtail adapter cable (widely available, inexpensive)
- Any standard 2.4 GHz Wi-Fi antenna with an SMA connector will then work with your build

 **Tip:** The U.FL connector is small and fragile. Connect and disconnect external antennas carefully — the connector is rated for a limited number of mate/demate cycles. Once your antenna is set up, leave it connected rather than repeatedly swapping.

Omnidirectional vs Directional Antennas

Omnidirectional (Duck / Dipole) Antennas

Omnidirectional antennas radiate and receive signal in a roughly equal pattern in all horizontal directions — like a donut shape surrounding the antenna. This makes them ideal for wardriving and general RF surveying where you want to detect networks from every direction as you move.

- Best for: Wardriving, general AP scanning, passive monitoring in all directions
- Typical gain: 2-5 dBi for standard duck antennas
- Form factor: The familiar stubby or flexible rubber antenna common on routers and access points

Directional Antennas (Yagi-Uda, Panel)

Directional antennas concentrate their gain into a narrow beam, trading omnidirectional coverage for significantly increased range in a specific direction. A Yagi-Uda antenna pointed at a building across the street can capture handshakes and beacon frames at distances that would be completely invisible to a standard duck antenna.

- Best for: Long-range authorized assessments of a specific target location, highly directional surveys
- Typical gain: 8-15 dBi or more for Yagi antennas
- Trade-off: You must physically aim the antenna at the target — coverage in other directions drops sharply

 **Increased antenna gain extends your reach significantly. With greater reach comes greater responsibility to ensure you are only capturing traffic from**

networks you are authorized to assess. A Yagi antenna pointed down a street will detect networks across a wide geographic area.

Understanding dBi — More Is Not Always Better

dBi (decibels relative to an isotropic radiator) measures antenna gain — how much the antenna concentrates signal in its intended direction compared to a theoretical perfect sphere. Higher dBi means more gain, but the shape of the radiation pattern changes in ways that can actually hurt you in some scenarios.

A high-gain omnidirectional antenna compresses its donut-shaped pattern into a flatter, more horizontal disc. The gain in the horizontal plane increases, but signal directly above or below the antenna — devices on floors directly above or below you in a multi-storey building — becomes weaker. For a ground-floor wardriving setup this is usually fine. For a fixed monitoring station in a multi-floor environment it can cause gaps in coverage.

- 2-3 dBi: Standard gain, full donut pattern, best for general indoor use
- 5-7 dBi: Moderate gain, slightly compressed pattern, good for outdoor wardriving
- 9+ dBi: High gain, very flat pattern, best for long-range horizontal coverage only

 **Tip:** For most wardriving builds, a 5 dBi omnidirectional antenna is the sweet spot — meaningful range improvement over the PCB antenna without the pattern distortion of very high gain antennas. Save the Yagi for specific authorized long-range assessments.

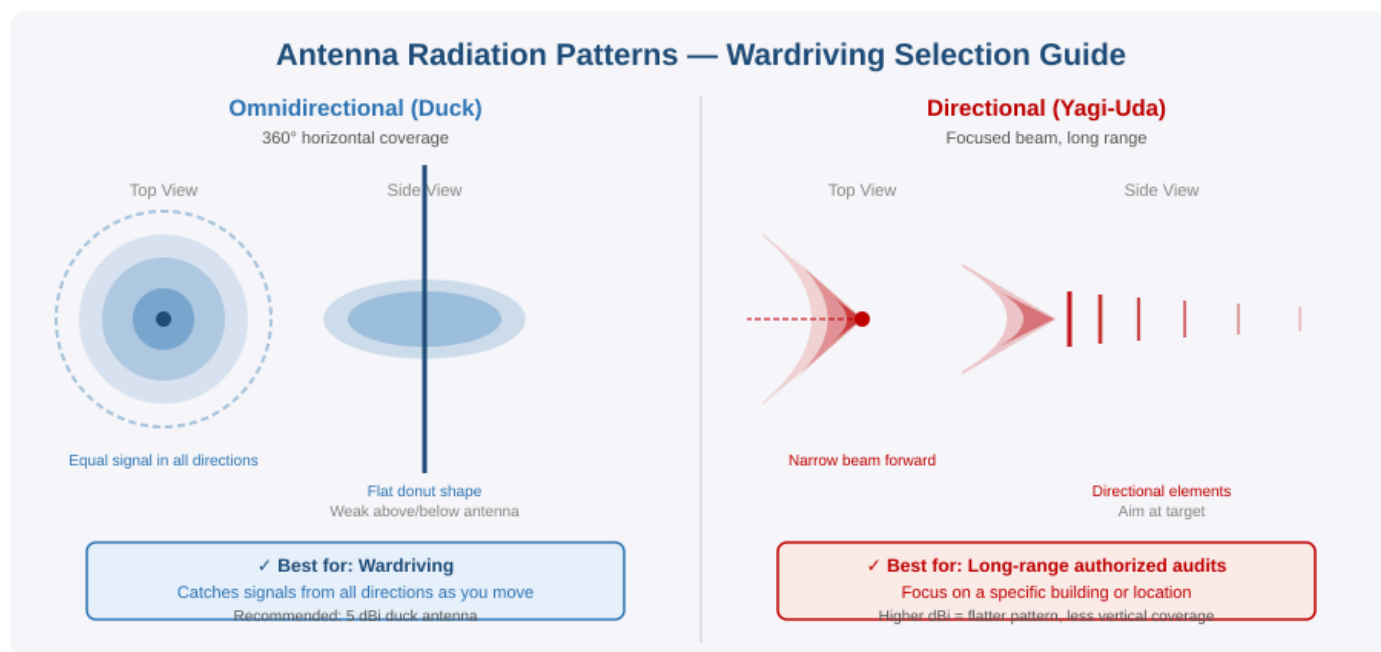


Figure 4 — Omnidirectional vs Yagi-Uda radiation patterns: choosing the right antenna for the job

Legal Note on Antenna Modifications

⚠ Radio frequency regulations in Belgium, the EU, and most countries impose limits on transmit power and antenna gain for unlicensed 2.4 GHz devices. Connecting a high-gain antenna to an ESP32 may cause the effective radiated power to exceed legal limits for unlicensed operation. For passive receive-only use (scanning and sniffing) this is generally not an issue — you are not transmitting. For active functions (beacon spam, deauth, evil portal) using a high-gain antenna, research your local ETSI and BIPT regulations before proceeding.

Building a War-Box: The Persistent Monitoring Station

A handheld Marauder device is excellent for casual wardriving and on-the-go scanning. But for serious extended sessions — long wardriving routes, persistent RF monitoring, or automated data pipelines — a dedicated 'War-Box' takes things to the next level.

A War-Box is a ruggedised, self-contained enclosure housing your Marauder device, a large-capacity battery, a GPS module, and optionally an external antenna — all wired together and ready to run unattended for hours.

The Core Build

Enclosure

The enclosure is the heart of a War-Box build. It needs to be robust enough to survive being thrown in a bag or mounted in a vehicle, water-resistant enough for outdoor use, and spacious enough to house the battery and electronics without heat buildup.

- Pelican-style cases: The gold standard for ruggedised builds. Waterproof, crushproof, and available in sizes from a wallet to a briefcase. The Pelican 1010 or equivalents are popular for compact Marauder builds.
- 3D-printed enclosures: If you have access to a printer (like the EasyThreeD K9), a custom-designed enclosure lets you place components exactly where you want them, add mounting points for antennas, and fit the battery precisely. PLA works fine for indoor use; PETG or ASA are better choices for outdoor builds exposed to temperature variation.
- Hammond project boxes: Affordable, widely available, and easy to drill and modify. Not as ruggedised as Pelican but perfectly serviceable for a first War-Box.

Battery

For a War-Box designed for extended sessions, a larger battery than a standard handheld build is essential. A 10,000mAh LiPo or flat lithium pack gives you 8-12 hours of active wardriving depending on the build's current draw. Combined with a quality TP4056-based charge controller and protection circuit, this becomes a reliable long-run power source.

- 10,000mAh LiPo flat pack: 8-12 hours runtime, fits in a Pelican 1010-size case
- USB power bank (passthrough capable): An alternative approach — use a quality 10,000mAh+ USB power bank that supports passthrough charging, so the device continues running while plugged in

💡 Tip: If using a USB power bank as the power source, test it specifically for passthrough operation — not all power banks continue output while charging. A power bank that cuts output when plugged in is useless for a fixed monitoring station.

GPS Module

For a War-Box, the NEO-8M is worth the small upgrade over the NEO-6M. Its faster acquisition time and better performance in challenging urban environments (where tall buildings can partially block satellite signals) make it more reliable for sustained automated wardriving sessions where you cannot afford to wait for a fix.

External Antenna

A War-Box is the ideal application for an external antenna. Mount a 5-7 dBi omnidirectional antenna on the exterior of the enclosure via an SMA bulkhead connector — this gives you much better range than an internal PCB antenna enclosed in a case, and allows the antenna to be positioned optimally (upright, unobstructed) regardless of how the box is mounted.



Figure 5 — War-Box component overview and automated data pipeline

Automated Data Pipeline

One of the biggest advantages of a War-Box over a handheld device is the ability to build an automated workflow that processes your captured data without manual intervention.

Capture Phase — On the Device

Set up Marauder or HaleHound for automated logging before you head out. With GPS fix confirmed and wardriving logging active, the device captures and writes to the SD card continuously for the entire session without requiring any interaction.

Sync Phase — Back at Base

When you return, remove the SD card and insert it into your computer. A simple Python script can automate the transfer of new files to a dedicated processing directory:

```
import shutil, os, glob
SD_PATH = '/Volumes/MARAUDER/wardrive/'
DEST_PATH = '~/wardriving/sessions/'
for f in glob.glob(SD_PATH + '*.csv'):
    shutil.copy(f, os.path.expanduser(DEST_PATH))
```

 **Tip:** Adapt the SD_PATH to match your SD card mount point — on Windows it will be a drive letter like E:\wardrive\, on macOS /Volumes/MARAUDER/wardrive/, and on Linux something like /media/username/MARAUDER/wardrive/.

Visualisation Phase — WiGLE & GPS Visualizer

Once synced to your base station, upload CSV logs to WiGLE for global leaderboard contribution and mapping. For local visualisation of your specific routes and coverage, drop the same CSV files into GPS Visualizer to see exactly where you went and how many networks you detected along each stretch.

If you are running a home server (Unraid, TrueNAS, a Raspberry Pi, or similar), consider setting up a watched folder that automatically processes new wardriving CSV files and generates a local map — a satisfying automated pipeline that turns each session into a visual report without manual effort.

Vehicle Mounting

A War-Box designed for vehicle-based wardriving benefits from a few additional considerations:

- **Magnetic base antenna mount:** A magnetic SMA antenna mount lets you place a 5-7 dBi antenna on the car roof for maximum GPS and Wi-Fi visibility, connected via a run of coax through a window seal.
- **12V to USB power:** A quality 12V to USB adapter (not a cheap charger) powers the War-Box from the vehicle's electrical system for unlimited runtime on long routes.
- **Secure mounting:** Use a RAM mount, Velcro strips, or a custom bracket to keep the box from sliding around. A rattling War-Box is a damaged War-Box.
- **Heat management:** Electronics in a closed box in a parked vehicle in summer can get very hot. If leaving the box in a vehicle unattended, ensure ventilation or use a case with thermal mass. The ESP32 thermal shutdown threshold is around 85°C.

Firmware Update Guide

Keeping Marauder firmware up to date ensures you have the latest features, bug fixes, and hardware support. The update process is straightforward once you have done it once — but doing it wrong can leave your device unresponsive. Read this section fully before flashing anything.

The Golden Rule: Always Match Firmware to Hardware

⚠️ Flashing the wrong firmware binary for your hardware is the single most common cause of a bricked or misbehaving Marauder device. Always confirm your exact board variant before downloading any firmware file.

Marauder firmware releases include multiple .bin files — one for each supported hardware variant. These are NOT interchangeable. A binary built for a MakerFocus ESP32 with an ILI9341 display will not work correctly on a raw ESP32 DevKit, and vice versa. At best you will get a blank screen. At worst the device will crash-loop on boot.

How to Identify Your Hardware

If You Bought a Prebuilt Device

Check the product listing, receipt, or box. The device name should match one of the hardware targets listed in the official GitHub release notes. If unsure, check JustCallMeKoko's GitHub — the README lists supported devices and their corresponding firmware file names.

If You Built Your Own

You need to know: (1) your ESP32 chip variant (ESP32, ESP32-S2, ESP32-S3), (2) your display driver (ILI9341, ST7789, or none), and (3) whether your build includes SD card and GPS. The firmware filename in the release encodes these details — read each release's description carefully before downloading.

 **Tip:** When in doubt, ask in the community Discord with a photo of your hardware. Someone will identify it in minutes and point you to the right binary.

Before You Flash: Safety Checklist

- Confirm your exact hardware variant and download the matching .bin file.
- Use a data-capable USB cable, not a charge-only cable. Test with a known-good cable if unsure.
- Back up any files on your SD card to your computer before flashing.
- Make sure no other application (Arduino IDE, serial monitor) is using the device's COM port.
- Charge your device if it is battery-powered — do not flash on low battery.
- Read the release notes for your target version. Some updates include breaking changes to SD card file structure or CLI commands.

Method 1: ESP32 Marauder Web Flasher (Recommended for Beginners)

The community-maintained web flasher is the simplest and safest option for most users. It runs entirely in your browser using the Web Serial API — no software installation required.

- Step 1: Open Google Chrome or Microsoft Edge. The Web Serial API is not supported in Firefox or Safari.
- Step 2: Navigate to the official Marauder web flasher (linked from the GitHub repository).
- Step 3: Connect your ESP32 device to your computer via USB.
- Step 4: Click 'Connect' and select your device's COM port from the popup list.
- Step 5: Select your exact device type from the dropdown. If your device is not listed, stop and verify your hardware before continuing.
- Step 6: Select the firmware version. For most users, choose the latest stable release.
- Step 7: Click 'Flash' and wait. Do not disconnect the device during flashing (typically 1-3 minutes).
- Step 8: When complete, disconnect and reconnect the device to boot into the new firmware.

 **Tip:** If your device is not detected by the web flasher, try a different USB cable first. Charge-only cables have no data lines and are the most common cause of detection failure.

Method 2: ESPTool (Command Line — Advanced)

ESPTool is Espressif's official command-line flashing utility. More flexible than the web flasher and works on any platform with Python installed. Recommended for users comfortable with a terminal.

Install ESPTool

```
pip install esptool
```

Erase Flash First (Recommended for Clean Install)

Before flashing a new version, especially across major releases, erasing the full flash first prevents stale data from causing boot issues:

```
esptool.py --chip esp32 --port COM3 erase_flash
```

Flash the Firmware

```
esptool.py --chip esp32 --port COM3 write_flash -z 0x1000 marauder_firmware.bin
```

Replace COM3 with your actual port (COMx on Windows, /dev/ttyUSB0 or /dev/tty.usbserial-* on macOS/Linux). Replace esp32 with esp32s2 or esp32s3 if using those variants.

 **If flashing fails with 'Failed to connect', hold the BOOT button on your ESP32 while running the flash command, then release it once you see 'Connecting...' in the terminal. Some boards require this to enter flash mode manually.**

What To Do If Your Device Appears Bricked

A 'bricked' ESP32 almost always means the firmware flashed incorrectly or the wrong binary was used. Unlike some devices, ESP32s are extremely difficult to permanently brick — there is almost always a recovery path.

Step 1: Do Not Panic

If your device boots to a blank screen, crash-loops, or is completely unresponsive, it is almost certainly recoverable. The ESP32's ROM bootloader is stored in read-only memory and cannot be overwritten by a user flash operation.

Step 2: Enter Manual Flash Mode

Hold the BOOT button, press and release the RST (reset) button, then release BOOT. The device is now in download mode and ready to be reflashed regardless of what is currently in flash memory.

Step 3: Full Erase and Reflash

With the device in download mode, run a full flash erase followed by a fresh flash of the correct firmware binary:

```
esptool.py --chip esp32 --port COM3 erase_flash
esptool.py --chip esp32 --port COM3 write_flash -z 0x1000 CORRECT_firmware.bin
```

Step 4: Verify the Binary

Double-check that the firmware file you are flashing is the correct one for your hardware. Read the release notes. Cross-reference the filename with the device list in the GitHub README. Flashing the right file after a full erase resolves the vast majority of 'bricked' cases.

 **Tip:** If you are still stuck after a full erase and reflash with the verified correct binary, post in the community Discord with your hardware details, the exact firmware filename you used, and the error output from ESPTool. The community is very helpful and most hardware issues are solved quickly.

Checking Your Current Firmware Version

On-device: Navigate to the About or System Info screen in the Marauder UI (exact location varies by firmware version).

Via serial console: Connect at 115200 baud and send the 'info' command to display the firmware version, build date, and hardware target.

Should You Always Update to the Latest?

Not necessarily. Newer is not always better for every use case:

- Latest stable: Best for most users. Gets all bug fixes and new features with reasonable stability testing.
- Previous stable: Worth keeping if your current version works perfectly and a new release has known regressions reported by the community.
- Beta/pre-release: Only if you want to test new features and are comfortable with occasional instability. Always note your previous version before updating so you can roll back.

💡 Tip: Keep a copy of a known-working firmware binary on your computer so you can always roll back quickly if a new release causes problems.

Firmware Alternatives: Bruce & HaleHound

Marauder is the most feature-rich and widely supported firmware for ESP32 wireless analysis — but it is not the only option. The open-source community has produced alternative firmware projects that run on the same hardware and offer different strengths. Understanding what each brings to the table helps you choose the right tool for the job, and gives you options as your skills grow.

Bruce Firmware

Bruce is a community-developed alternative firmware that takes a different approach to the ESP32's capabilities. Where Marauder focuses almost exclusively on Wi-Fi and BLE, Bruce casts a wider net.

What Makes Bruce Different

- Icon-based UI: Bruce uses a graphical icon-driven interface rather than Marauder's text menu. Many users find it more intuitive to navigate, especially on smaller screens.
- Sub-GHz support: If your build includes a CC1101 Sub-GHz radio module, Bruce can interact with Sub-GHz frequencies — giving you access to remote controls, weather stations, and other 433/868/915 MHz devices. This is a capability Marauder does not have natively.
- Built-in game: Bruce includes a small built-in game — a nice touch that makes it feel more like a genuine portable device than a pure security tool.
- IR transmitter support: If your build includes an infrared LED, Bruce can function as a universal remote control.
- Broader protocol coverage: Beyond Wi-Fi, Bruce supports NFC (with compatible hardware), RFID, and Bluetooth Classic in addition to BLE.

When to Consider Bruce

Bruce is an excellent choice if you want a single device that does more than just Wi-Fi analysis — particularly if you have or plan to add a CC1101 module for Sub-GHz experimentation. It is also worth trying if you find Marauder's text UI less intuitive.

Flashing Bruce follows the same process as flashing Marauder — you need a device with proper USB serial access. Check the Bruce GitHub repository for supported hardware and the correct binary for your build.

💡 Tip: Bruce and Marauder can coexist on the same device if you are comfortable reflashing. Many enthusiasts keep both — switching between them depending on whether they are doing Wi-Fi analysis (Marauder) or broader multi-protocol experimentation (Bruce).

HaleHound

HaleHound takes a fundamentally different philosophy: it is optimised for speed, automation, and sustained background operation rather than interactive use.

What Makes HaleHound Different

- Set-it-and-forget-it wardriving: HaleHound is built for high-speed, automated wardrive logging with minimal user interaction. You set it up, put the device in your bag or vehicle, and it captures as fast as the hardware allows.
- Optimised capture speed: HaleHound prioritises throughput — logging as many networks as possible as quickly as possible — over a rich interactive UI.
- Minimal screen dependency: HaleHound is less focused on the display and more focused on the SD card. It works well in headless or near-headless configurations.
- Automated logging pipeline: Designed to integrate cleanly with post-processing scripts and wardriving data pipelines.

When to Consider HaleHound

HaleHound makes most sense for dedicated wardriving rigs where the goal is maximum coverage in minimum time. If you are building a War-Box (see the next section) designed to run unattended in a vehicle, HaleHound's automated logging approach may suit that use case better than Marauder's interactive interface.

Choosing Between Them

There is no wrong answer — the three firmwares serve different purposes and many enthusiasts flash different ones depending on the task at hand. As a starting point: begin with Marauder to learn the fundamentals, experiment with Bruce once you want to explore beyond Wi-Fi, and consider HaleHound when you are ready to build a dedicated automated wardriving rig.

⚠ Flashing any alternative firmware requires reliable USB serial access to your device. On clone hardware without a proper USB port, switching between firmware projects may require a UART adapter and direct connection to programming pads. Always verify your hardware has accessible flashing capability before committing to a firmware ecosystem.

SD Card Setup & File Structure

The SD card is Marauder's external memory — it stores your PCAP captures, wardriving logs, GPS data, Evil Portal HTML files, and configuration. Getting it set up correctly from the start prevents a lot of frustration.

Initial SD Card Setup

- Step 1: Use a card 32GB or smaller. Larger cards are often formatted as exFAT by default, which Marauder does not support.
- Step 2: Format the card as FAT32. On Windows, right-click the drive in File Explorer → Format → FAT32. On macOS, use Disk Utility → Erase → MS-DOS (FAT). On Linux, use `mkfs.fat -F 32`.
- Step 3: Insert the card into your Marauder device before powering on.
- Step 4: Power on the device. Marauder will automatically create its directory structure on first boot.

 **Tip:** On Windows, cards larger than 32GB cannot be formatted as FAT32 using the built-in formatter. Use a free third-party tool like Rufus or FAT32 Format to handle larger cards if needed.

Directory Structure

After first boot with a fresh SD card, Marauder creates the following folder structure:

```
/
├── pcaps/           - Saved PCAP capture files
├── wardrive/        - Wardriving log files (.csv)
├── portals/         - Evil Portal HTML files
└── logs/           - General activity logs
```

PCAP File Naming

Marauder names PCAP files automatically using a timestamp or sequential number depending on firmware version. A typical filename looks like:

```
pcap_20240315_142301.pcap
```

The format is `pcap_YYYYMMDD_HHMMSS.pcap` where the date and time reflect when the capture was started. This makes it easy to correlate captures with notes or events in your log.

Wardriving Log Format

Wardriving logs are saved as CSV files compatible with WiGLE and other wardriving platforms. Each row contains:

- Timestamp of detection
- SSID and BSSID of the detected network
- RSSI (signal strength)
- Channel and encryption type
- GPS latitude, longitude, altitude, and accuracy

 **Tip:** Before uploading to WiGLE, open your CSV in a spreadsheet application and do a quick sanity check: confirm GPS coordinates are valid (not 0.0, 0.0) and that entries contain the expected fields.

Evil Portal Files

Custom HTML pages for the Evil Portal feature are stored in the /portals/ directory. You can create your own portal pages and copy them to this folder via a card reader. The file must be named index.html or as specified by your firmware version's documentation.

Managing Storage

- Regularly transfer captures to your computer and clear the SD card to prevent it filling up mid-session.
- A full SD card will cause Marauder to silently fail to save new captures — check available space before long wardriving sessions.
- Use the SD Card Browser in the Marauder UI to view and delete files directly on the device without removing the card.

WiGLE & Wardriving Platforms Deep Dive

WiGLE (Wireless Geographic Logging Engine) is the largest community-driven wireless mapping platform in the world. Founded in 2001, it has grown into a global database of billions of mapped Wi-Fi and Bluetooth networks contributed by thousands of wardrivers.

Creating a WiGLE Account

- Step 1: Visit wiggles.net and click 'Register' to create a free account.
- Step 2: Verify your email address.
- Step 3: Log in and navigate to your profile to set your display name and preferences.

 **Tip:** Your WiGLE username becomes your identity on the leaderboards and in team competitions. Choose something you are happy to be known by in the community.

Uploading Marauder Wardriving Logs

- Step 1: Transfer your wardriving CSV logs from the SD card to your computer.
- Step 2: Log in to WiGLE and click 'Upload' in the top navigation.
- Step 3: Select your CSV file(s) and click Upload.
- Step 4: WiGLE will process your submission — this can take a few minutes for large files.
- Step 5: Once processed, your networks will appear on the WiGLE map and your stats will update.

 **Review WiGLE's terms of service and privacy policy before uploading. Be aware that uploaded data becomes part of a public database. Do not upload data you are not comfortable sharing.**

Reading Your WiGLE Stats

Your WiGLE profile page shows a detailed breakdown of your contributions:

- Total networks discovered: Cumulative unique networks you have logged across all uploads.
- Networks discovered this month/year: Useful for tracking active wardriving periods.
- Unique networks: Networks you discovered that were not previously in the WiGLE database — these count extra for leaderboard ranking.
- Rank: Your position on the global leaderboard based on unique network discoveries.
- Map coverage: A visual heatmap of your wardriving routes and coverage.

The WiGLE Leaderboard & Rankings

WiGLE maintains global and monthly leaderboards ranking contributors by unique network discoveries. The system is designed to reward exploration of new areas rather than simply re-scanning already-mapped locations.

- Discoverer rank: Based on total unique networks first logged by you in the WiGLE database.
- Monthly rank: Resets each month — great for friendly competition.
- Country rank: How you rank against wardrivers in your specific country.

 **Tip:** The fastest way to climb the leaderboard is to wardrive in areas with little prior coverage. Rural roads, small towns, and industrial areas are often unmapped and yield high proportions of unique networks.

Joining a WiGLE Team

Teams are groups of wardrivers who pool their discoveries for collective ranking. Joining a team is a great way to connect with the community, participate in group challenges, and compare your progress with like-minded enthusiasts.

- Browse teams on the WiGLE website under the 'Teams' section.
- Request to join an existing team or create your own.
- Team leaderboards show combined unique discovery counts.

Other Wardriving Platforms

OpenWiFiMap

A community-driven alternative to WiGLE focused on open data. Accepts similar CSV/PCAP uploads and contributes to an openly licensed wireless map dataset.

Wifileaks

Another wardriving aggregator with its own mapping and statistics features. Less active than WiGLE but worth knowing as an alternative upload destination.

Troubleshooting

Here are the most common issues Marauder users encounter, along with practical solutions. If your issue is not listed here, the official GitHub issues page and community Discord are excellent next steps.

Flashing & Boot Issues

Device not recognized by computer when connected via USB

- Try a different USB cable — charge-only cables (common with phone chargers) have no data lines and will not work.
- Install the correct USB-to-serial driver for your board. Most ESP32 boards use either the CP2102 or CH340 chip. Search for the driver that matches your board.
- Try a different USB port or a powered USB hub.
- On Windows, check Device Manager for an unknown device — this will help identify which driver is missing.

Flash fails with 'Failed to connect to ESP32' error

- Hold the BOOT button on your ESP32 while clicking Flash, then release it once the flashing process begins.
- Some boards require you to briefly press the RST (reset) button while holding BOOT to manually enter flash mode.
- Reduce the flash baud rate in ESPTool using `--baud 115200` if higher speeds are causing errors.
- Check that no other application (serial monitor, Arduino IDE) is using the COM port simultaneously.

Device boots but shows blank or garbled screen

- The display may not be initialized for your specific screen driver (ILI9341 vs ST7789). Ensure you flashed the correct firmware binary for your hardware variant.
- Check your display wiring against the pinout diagram for your specific board and firmware version.
- Try a full erase before reflashing: `esptool.py --chip esp32 erase_flash`, then flash again.

GPS Issues

GPS Monitor shows no fix or 0 satellites

- Ensure your GPS module is outdoors or near a window with clear sky visibility. GPS signals do not penetrate buildings well.
- Allow 1-2 minutes for cold start acquisition. First fix after a long time without power takes longer.

- Verify your GPS module is wired to the correct UART pins for your firmware build. Check the pinout in the official documentation.
- Confirm the GPS module is powered correctly — most modules need 3.3V or 5V depending on the model.

GPS fix obtained but coordinates look wrong

- A single-satellite fix can produce inaccurate coordinates. Wait for at least 4 satellites for reliable positioning.
- Check that your GPS module's baud rate (usually 9600) matches the UART baud rate configured in your firmware.

SD Card Issues

SD card not detected

- Confirm the card is formatted as FAT32 — not exFAT or NTFS.
- Try a different card. Some SD cards have compatibility issues with the ESP32's SPI interface.
- Check SD card wiring if using a DIY build. SPI connections (MOSI, MISO, SCK, CS) must all be correct.
- Insert the card before powering on the device, not while it is running.

PCAP files not being saved

- Check available space on the SD card — a full card will silently fail to write new files.
- Verify the /pcaps/ directory exists. If not, create it manually from a computer.
- Some firmware versions require Save PCAP to be explicitly enabled before a scan — check the settings menu.

Wardriving logs missing GPS data (coordinates showing 0.0, 0.0)

- This almost always means there was no GPS fix when logging started. Always verify a GPS fix in the GPS Monitor screen before beginning a wardriving session.
- Check that your GPS module's TX pin is connected to the correct ESP32 RX pin for your firmware build.

Scanning Issues

AP Scan shows no networks or very few networks

- Ensure Wi-Fi is enabled in Marauder settings.
- Move to a location with known Wi-Fi coverage to verify the scanner is working.
- Some firmware builds require you to explicitly start a scan — check that you have initiated the scan and not just opened the menu.

Station Scan shows no clients

- Station Scan requires at least one AP to be detected first. Run AP Scan before Station Scan.
- Clients may not appear immediately — allow 10-30 seconds for passive detection.
- Clients that are idle or in power-save mode transmit less frequently and may take longer to appear.

Probe Sniff capturing nothing

- Most modern devices randomize their MAC addresses and may not send probe requests with known SSIDs. Try with an older device or one with MAC randomization disabled.
- Move to a busier area — more devices means more probe requests.

Performance Issues

Device freezing or rebooting during use

- Check battery voltage — low battery can cause unexpected resets, especially under the load of GPS + Wi-Fi + SD write simultaneously.
- Ensure your SD card is Class 10 or faster. Slow cards can cause write timeouts that destabilize the firmware.
- Try flashing the latest firmware — stability bugs are common targets for updates.

Screen flickering or display artifacts

- Often a power supply issue — ensure your battery or USB power source can deliver adequate current.
- Check display wiring for loose connections if on a DIY build.

The Wall of Shame — Classic Pitfalls

These are the mistakes that trip up newcomers most reliably. Each one has cost someone hours of frustration. Learn from them so you do not have to repeat them.

The Brownout Reboot Loop

Symptom: Your Marauder powers on fine, boots to the menu, but the moment you start a Wi-Fi scan the device immediately reboots — over and over.

Cause: The ESP32's radio can spike to 500mA or more during active scanning. If your USB cable, power bank, or battery cannot deliver that current cleanly, the supply voltage drops below the ESP32's brownout detection threshold and the chip resets itself as a safety measure.

- Try a different USB cable. Thin or low-quality cables have high resistance and cannot deliver peak current. Use a short, thick, high-quality cable rated for data and charging.
- Try a different power bank or charger. Some power banks have low-current modes that throttle output — particularly older ones or ultra-slim models.

- If on a DIY battery build, check the TP4056 charge controller and wiring. Undersized wires between the battery and the ESP32 are a common culprit.
- Reduce simultaneous load — if GPS, display backlight, and SD card writes are all active at once during a scan, total current draw is at its peak. Try reducing display brightness first.

💡 Tip: A 500mA capable USB port or power bank is a minimum. For a full build with GPS, display, and SD card all active, aim for a source capable of delivering 1A comfortably.

The Fake SD Card Trap

Symptom: Your SD card appears to work — Marauder detects it, the directory structure looks correct, and captures seem to save. But after a wardriving session or extended PCAP capture you find the files are empty, corrupt, or stop after a certain size.

Cause: Counterfeit SD cards are extremely common on budget marketplaces. They report a large capacity (32GB) to the operating system but physically contain much less storage (often 4GB or even 512MB). Once writes exceed the real physical capacity, data silently corrupts or disappears.

- Use H2testw (Windows, free) or F3 (Linux/macOS, free) to verify every SD card before trusting it with important captures. These tools write test data across the full reported capacity and verify what was written — exposing fake cards immediately.
- Buy SD cards from reputable retailers rather than unknown marketplace sellers. Genuine SanDisk, Samsung, and Kingston cards from official channels are worth the small premium.
- Always check the card after a long session. Open the SD Card Browser on the device and verify file sizes look reasonable before assuming the capture was successful.

💡 Tip: H2testw is a free download from the heise.de website. Run it on every new SD card before first use. It takes 30-60 minutes for a full 32GB verification but will save you from losing hours of wardriving data to a counterfeit card.

The Charge-Only Cable

Symptom: You plug the device into your computer and nothing happens. No COM port appears. The web flasher cannot find the device. ESPTool reports no serial port.

Cause: Many USB cables — particularly the ones bundled with cheap electronics — are wired for charging only. They carry power (5V and GND) but have no data lines (D+ and D-). Your computer literally cannot see the device because there is no data connection.

- Test your cable by plugging in a known-working USB device that requires data (a phone in file transfer mode, for example). If your computer does not recognise it, the cable is charge-only.
- Use a cable explicitly rated for data transfer. USB-A to Micro-USB or USB-C cables sold as 'data + charge' cables work reliably.
- Keep one verified data-capable cable dedicated to your Marauder and label it. It will save confusion every time you need to flash firmware.

Wrong Firmware Binary

Symptom: Flashing completes without errors but the device boots to a blank screen, shows garbled graphics, or crashes on startup.

Cause: You flashed the correct chip (ESP32) but the wrong hardware variant. The firmware binary contains pin definitions for a specific display, SD card wiring, and button layout. Flash the wrong one and those definitions point to the wrong pins on your board.

- Return to the GitHub releases page and read the description of each binary carefully. Match your display driver (ILI9341 vs ST7789), your board variant, and any special hardware features.
- When genuinely unsure which binary to use, post a photo of your hardware in the community Discord. Someone will identify it in minutes.
- Always do a full flash erase before reflashing a different binary: `esptool.py erase_flash`. Stale data from the previous firmware can cause unpredictable behaviour even after flashing a new binary.

GPS Coordinates of 0.0, 0.0

Symptom: Your wardriving log looks perfect — lots of networks detected — but every single entry has latitude 0.000000 and longitude 0.000000.

Cause: Logging started before a GPS fix was acquired. Marauder logs the GPS coordinates it has at the moment of each network detection. If no fix has been obtained yet, it logs zeros as a placeholder.

- Always check the GPS Monitor screen before starting a wardriving session. Wait until satellite count is at least 4 and coordinates look correct for your location.
- If you are indoors or in a dense urban environment, move to a window or step outside briefly to acquire the initial fix. Once locked, you can often return indoors and the fix will hold for a while.
- WiGLE will reject or discard entries with 0.0 coordinates — so this mistake does not pollute the database, but it does mean your session data is lost.

Mid-Session Error Reference

These are the failures that happen not during setup, but while you are actively using the device. Unlike first-boot problems, mid-session errors often have no obvious error message — Marauder just stops doing what you expected. This reference covers the most common patterns and their causes.

Scan starts but no networks appear

Expected: Networks populate within a few seconds of starting AP Scan.

Actual: The scan appears to run but the list stays empty.

- Confirm Wi-Fi scanning is enabled in the Marauder settings menu — some firmware versions default it to off.
- Try moving to a different room or outdoors. If you are in a basement or surrounded by thick concrete, Wi-Fi signals may be genuinely absent.

- Verify the scan has actually started — some firmware versions require an explicit confirm press after selecting the scan type.
- Restart the device and try again. A failed initialisation of the Wi-Fi radio occasionally causes the first scan attempt to produce no results.
- Check your firmware version. Some older or unofficial builds have known scanning bugs. Update to the latest stable release.

AP Scan shows results but Station Scan shows nothing

Expected: Client devices associated with detected APs appear within 30-60 seconds.

Actual: Station Scan runs indefinitely with no entries.

- Station Scan is passive — it waits for client devices to transmit. Devices in deep power-save mode may not transmit for minutes at a time. Wait longer.
- Move closer to the AP. Clients associated with a far AP may be transmitting at a power level too low for Marauder to receive reliably.
- Try using your test device — open a browser or stream video to force it to transmit actively, making it easier for Marauder to detect.
- Confirm you ran AP Scan first. Some firmware versions require the AP list to be populated before Station Scan can associate clients to networks.

PCAP file not created or file is empty / 0 bytes

Expected: A .pcap file appears in /pcaps/ on the SD card after capturing.

Actual: No file exists, or the file exists but contains no data.

- Confirm Save PCAP is enabled in settings before starting the scan — it is not always on by default.
- Check available space on the SD card via the SD Card Browser. A full card silently fails to write.
- Verify the /pcaps/ directory exists on the card. Create it manually from a computer if missing.
- Test the SD card with H2testw (Windows) or F3 (Linux/macOS) to rule out a counterfeit card with misreported capacity.
- Try a different SD card. Some cards are incompatible with the ESP32's SPI write speed even if they pass capacity tests.

Serial console connects but commands produce no output

Expected: Typing a command in the terminal and pressing Enter produces a response from Marauder.

Actual: Commands appear to be accepted but nothing happens.

- Confirm your terminal's line ending is set to 'Both NL & CR' (or CRLF). Commands sent with LF only are often silently ignored by Marauder's CLI parser.
- Check the baud rate is set to exactly 115200. A mismatched baud rate produces either garbled output or silence.

- Try sending the 'help' command. If that produces output, your connection is working but the specific command syntax may have changed in your firmware version.
- Some terminal applications buffer input — try pressing Enter twice after your command.
- Restart the device while the serial connection is active and observe the boot log. If the boot log appears correctly, the connection is good.

Device detected by computer but web flasher cannot connect

Expected: Clicking Connect in the web flasher shows your device's COM port and allows selection.

Actual: No COM port appears in the popup, or the popup is empty.

- Confirm you are using Google Chrome or Microsoft Edge. Firefox and Safari do not support the Web Serial API.
- Close any other application that may have the serial port open — Arduino IDE Serial Monitor, PuTTY, minicom. Only one application can hold a serial port at a time.
- Try a different USB cable and a different USB port on your computer.
- On Windows, check Device Manager for the COM port. If it shows a warning triangle, reinstall the CP2102 or CH340 driver.
- Try the ESPTool command-line method instead — it sometimes succeeds when the web flasher cannot negotiate the connection.

GPS fix acquired indoors but lost when wardriving begins

Expected: GPS fix maintained throughout the wardriving session.

Actual: Fix is lost shortly after leaving the initial location, or intermittently drops.

- The ESP32's GPS UART buffer can overflow if the firmware is not reading it fast enough during heavy simultaneous Wi-Fi scanning. This is a known limitation — reduce simultaneous active functions if possible.
- Check your GPS antenna orientation. The NEO-6M's ceramic patch antenna must face upward (toward the sky) for optimal reception. Inside a case, ensure no metal components are directly above the antenna.
- In dense urban areas, tall buildings create 'urban canyons' that block satellite signals. Gaps in fix are normal in these environments — choose routes with more open sky when possible.
- Verify the GPS module's power supply. A brownout-level voltage dip affects GPS module stability before it affects the ESP32 itself.

Device reboots mid-session without warning

Expected: Continuous operation throughout the session.

Actual: Device spontaneously reboots, sometimes repeatedly.

- The most common cause is brownout from insufficient current supply. See 'The Brownout Reboot Loop' in the Wall of Shame section above for the full diagnosis.
- Check for SD card write timeouts — a slow or failing SD card can cause firmware instability when the write queue backs up.

- Heat can cause instability in enclosed builds, particularly in warm vehicles. Ensure your enclosure has adequate ventilation.
- Check your firmware version. Some builds have known stability issues under specific conditions — the release notes and GitHub issues page list known regressions.
- If the reboot only occurs when a specific function is active (e.g. only during PMKID Sniff), report it as a potential firmware bug on the GitHub issues page with your hardware and firmware details.

Wireshark opens PCAP but shows 'file appears to be damaged'

Expected: PCAP opens cleanly and shows captured frames.

Actual: Wireshark reports a damaged or truncated file.

- This usually means the capture was interrupted — the device rebooted or the SD card was removed while writing was in progress. The file header was written but the data was truncated.
- Try opening the file with Wireshark anyway by clicking OK on the error — partial PCAPs often contain usable frames before the point of truncation.
- In Wireshark, go to File > Import from Hex Dump as an alternative approach for recovering data from a corrupted PCAP.
- Use the command-line tool pcapfix (available via `pip install pcapfix`) to attempt automatic repair of a truncated PCAP file.
- Prevention: Always stop the active scan and allow a few seconds for the SD card write buffer to flush before powering off the device.

Homelab Exercises

The best way to build confidence with Marauder is to use it — on your own equipment, in your own home, with no pressure and no risk. These exercises are designed to take you from first scan to advanced workflow, one step at a time. Each one has a clear goal, tells you what to expect on screen, and explains what the results actually mean.

Every exercise here is performed entirely on your own network and devices. No external networks are touched. No neighbours are affected. This is a safe, legal, and genuinely educational environment to develop real skills.

⚠ All exercises in this section must be performed on your own home network and devices only. Do not target any network or device you do not personally own.

What You Need for These Exercises

- Your Marauder device with a formatted SD card inserted
- Your home Wi-Fi router (any standard home router is fine)
- At least one Wi-Fi connected device — a phone, laptop, smart TV, or tablet
- A computer with Wireshark installed for the PCAP exercises (free at [wireshark.org](https://www.wireshark.org))
- Optional: a second phone or laptop to use as a dedicated test client

Exercise 1: Your First AP Scan — Reading the Wireless Landscape

Goal: Run your first AP Scan, understand what every field means, and identify your own router in the results.

Steps

- Step 1 — Power on your Marauder device and navigate to AP Scan in the menu.
- Step 2 — Start the scan and wait 10-15 seconds for results to populate.
- Step 3 — Look for your home network by SSID (the name you gave your Wi-Fi).
- Step 4 — Note down your router's BSSID, channel, RSSI, and encryption type.
- Step 5 — Count how many other networks appear that you do not recognise. These are your neighbours' networks.

What to Expect

In a typical home environment you should see anywhere from 5 to 30+ networks depending on how densely populated your area is. Your own router will likely show the strongest RSSI value (closest to 0 dBm) since you are physically near it. Neighbours' networks will show weaker signals.

What to Look For

- Your SSID and BSSID: Confirm Marauder correctly detected your router. Write down the BSSID — you will use it in later exercises.
- Encryption type: Confirm your network shows WPA2 or WPA3. If it shows WEP or Open, your network has a serious security problem that needs fixing immediately.
- Channel distribution: Note which channels the networks around you are using. If many networks share channel 1, 6, or 11 with yours, that is a source of interference and performance issues.
- Hidden networks: Any network with a blank SSID is configured to hide its name. It is still fully visible to Marauder — hiding an SSID provides no real security.

 **Tip:** Log in to your router's admin panel after this exercise and compare what Marauder detected with what your router reports about itself. They should match exactly. If the BSSID Marauder shows differs from what your router reports, you may have a mesh node or repeater broadcasting a slightly different MAC.

Exercise 2: Station Scan — Mapping Your Connected Devices

Goal: Discover every device connected to your home network and identify what each one is.

Steps

- Step 1 — Run an AP Scan first to populate the network list.
- Step 2 — Navigate to Station Scan and start it.
- Step 3 — Wait 30-60 seconds. Connected devices will appear progressively as they transmit.

- Step 4 — For each MAC address that appears, note which BSSID (router) it is associated with.
- Step 5 — Take the first six characters of each MAC address (the OUI) and look them up at macvendors.com to identify the manufacturer.
- Step 6 — Try to match every detected device to something physical in your home.

What to Expect

In a typical home you might see 10-30 devices: phones, laptops, smart TVs, games consoles, smart speakers, thermostats, robot vacuums, security cameras, and IoT sensors. Many households are surprised by how many devices are actually connected.

Interesting Things to Try

- Walk around your home while Station Scan is running. Devices in different rooms will have different RSSI values — you can roughly gauge which part of the house a device is in based on signal strength.
- Turn a device's Wi-Fi off and back on while Station Scan is active. Watch it disappear from the list and reappear.
- Check whether any devices appear that you cannot identify even after OUI lookup. An unrecognised device on your network is worth investigating — it could be a forgotten smart home gadget, or it could be something that should not be there.

💡 Tip: Cross-reference your Marauder station list with your router's DHCP client table (found in your router's admin panel). The router assigns IP addresses to each device and typically shows a hostname alongside the MAC — this makes identification much easier.

Exercise 3: Probe Sniff — Watching Your Devices Call Out

Goal: Capture probe requests from your own devices and observe what network names they are broadcasting.

Steps

- Step 1 — Start Probe Sniff on your Marauder device.
- Step 2 — Take your phone, turn Wi-Fi off, wait five seconds, then turn Wi-Fi back on.
- Step 3 — Watch the Marauder screen. Within seconds you should see probe requests appear from your phone's MAC address.
- Step 4 — Read the SSIDs being broadcast. These are the names of every saved Wi-Fi network on that phone.
- Step 5 — Repeat with a laptop or tablet for comparison.

What to Expect

Older phones and devices with MAC randomization disabled will broadcast a list of remembered SSIDs — home networks, work networks, hotel networks from past trips, coffee shop networks,

friends' houses. Every location where that device has ever connected to Wi-Fi may appear in the list.

Newer devices (iOS 14+, Android 10+) increasingly suppress specific SSIDs in probe requests and randomize their MAC address, so your results will vary by device age and OS version. This variation is itself instructive — it shows the privacy improvement in modern operating systems.

The Privacy Lesson

Show a family member or housemate what their phone is broadcasting. The reaction is almost always immediate — people do not realise their device is announcing a history of every network it has ever joined to anyone nearby with the right tools. This is one of the most impactful real-world privacy demonstrations you can run with Marauder, and it requires nothing beyond your own household devices.

 **Tip:** After this exercise, check your phone's Wi-Fi settings and clear old saved networks you no longer use. On iOS, go to Settings > Wi-Fi > Edit to remove old entries. On Android, long-press a saved network and choose Forget. Reducing the saved network list directly reduces probe request leakage.

Exercise 4: Beacon Sniff — Fingerprinting Your Router

Goal: Capture beacon frames from your own router and inspect the detailed information embedded in them.

Steps

- Step 1 — Start Beacon Sniff on your Marauder device.
- Step 2 — Let it run for 30 seconds near your router.
- Step 3 — With Save PCAP enabled, stop the capture and save the file to the SD card.
- Step 4 — Transfer the PCAP to your computer and open it in Wireshark.
- Step 5 — Filter by typing: `wlan.fc.type_subtype == 8` in the Wireshark filter bar. This shows only beacon frames.
- Step 6 — Click on a beacon from your router's BSSID and expand the 'IEEE 802.11 Wireless Management' section in the packet detail panel.

What to Look For in Wireshark

- SSID Parameter Set: Your network name, exactly as broadcast
- Supported Rates: The data rates your router advertises — gives you an idea of the Wi-Fi generation
- RSN Information (Robust Security Network): The exact encryption and authentication protocols in use — WPA2, CCMP, PSK etc.
- Vendor Specific tags: Manufacturer-specific data that can identify the router brand, model generation, and sometimes firmware family
- HT Capabilities / VHT Capabilities: Whether your router supports 802.11n or 802.11ac features

 **Tip:** The Vendor Specific information elements in beacon frames are a goldmine for router fingerprinting. Cisco, Netgear, TP-Link, and others all embed characteristic tags.

You can often identify not just the manufacturer but the product line from beacon data alone.

Exercise 5: Deauth Your Own Device — Understanding the Attack

Goal: Send a deauthentication frame to one of your own devices and observe what happens, then explore the defence.

⚠ This exercise must only be performed on your own devices connected to your own network. Sending deauth frames to any other network or device is illegal.

Background

A deauthentication attack works by sending forged management frames that tell a device its connection to the router has been terminated. Because older 802.11 standards do not require these frames to be cryptographically verified, the device accepts the instruction and disconnects — even though the frame came from an attacker rather than the real router. The classic use in a security context is to force a reconnection, capturing the WPA2 handshake that occurs when the device rejoins.

Steps

- Step 1 — Run AP Scan and note the BSSID of your home router.
- Step 2 — Run Station Scan and note the MAC address of a device you want to test — your own phone works well.
- Step 3 — Enable Save PCAP so you can review what happened afterward.
- Step 4 — Navigate to the deauth function in Marauder. Target your router's BSSID and your own device's MAC.
- Step 5 — Send a short burst of deauth frames (3-5 frames is sufficient for observation — you do not need a flood).
- Step 6 — Watch your target device. It should momentarily disconnect from Wi-Fi and then automatically reconnect within a few seconds.
- Step 7 — Stop the deauth and check the PCAP in Wireshark. Filter by 'eapol' to see the WPA2 four-way handshake that was captured during reconnection.

What You Just Observed

You forced your own device off the network and captured the authentication handshake it performed when rejoining. In an authorized security assessment context, this handshake material can be used with Hashcat to audit the strength of the Wi-Fi password. You have just completed, end-to-end, the core workflow of a WPA2 credential audit — entirely on your own equipment.

Now Test the Defence

Log in to your router's admin panel and look for a setting called Protected Management Frames, 802.11w, or PMF. Enable it and set it to Required if possible. Then repeat the deauth exercise. With PMF enabled, your device should either ignore the forged deauth frames entirely or be

significantly harder to disconnect — because the management frames are now cryptographically authenticated.

This is the single most important practical thing you can do with this exercise: not just seeing that the attack works, but verifying that your own router's defence against it is active.

 **Tip:** Not all routers support PMF, and some older client devices are incompatible with it. If enabling PMF breaks connectivity for some devices, try setting it to Optional rather than Required — this allows PMF-capable devices to use it while maintaining compatibility with older ones.

Exercise 6: Channel Analysis — Optimising Your Home Network

Goal: Use Marauder's channel information to identify Wi-Fi congestion in your area and improve your router's channel configuration.

Steps

- Step 1 — Run AP Scan and let it complete a full sweep.
- Step 2 — As results appear, note which channel each detected network is using.
- Step 3 — Count how many networks are on each channel — particularly channels 1, 6, and 11 in the 2.4 GHz band.
- Step 4 — Identify the least congested channel: the one with the fewest competing networks.
- Step 5 — Log in to your router's admin panel and change the 2.4 GHz channel to the least congested one.
- Step 6 — Run AP Scan again to confirm your router is now broadcasting on the new channel.

What to Expect

In a dense urban environment like an apartment building, you may find all three non-overlapping 2.4 GHz channels heavily occupied. In that case, pick the one with the fewest high-signal (strong RSSI) networks — weaker distant networks cause less interference than strong nearby ones.

The 5 GHz band has many more channels and is almost always less congested than 2.4 GHz. If your router supports 5 GHz and your devices support it, prefer 5 GHz for performance-sensitive devices and leave 2.4 GHz for IoT devices and anything that needs range.

 **Tip:** This exercise produces a tangible real-world improvement to your home network performance. Switching from a congested channel to a clear one can noticeably reduce latency and improve throughput — particularly for video calls and gaming. It is one of the most practically useful things you can do with a basic AP Scan.

Exercise 7: Full PCAP Capture & Wireshark Analysis

Goal: Capture a complete wireless session from your own network — including a WPA2 handshake — and explore it in detail in Wireshark.

Steps

- Step 1 — Enable Save PCAP in Marauder settings.
- Step 2 — Start Beacon Sniff or AP Scan to begin capturing frames.
- Step 3 — On a device connected to your home network, disconnect from Wi-Fi and reconnect. This forces a WPA2 handshake to occur.
- Step 4 — Let Marauder capture for 60 seconds, then stop and save.
- Step 5 — Transfer the PCAP to your computer and open in Wireshark.

Wireshark Filters to Try

- `eapol` — Shows WPA2 four-way handshake frames. You should see exactly 4 frames from the reconnection you triggered.
- `wlan.fc.type_subtype == 8` — Shows only beacon frames from nearby APs.
- `wlan.fc.type_subtype == 4` — Shows probe request frames from client devices.
- `wlan.fc.type_subtype == 12` — Shows deauthentication frames.
- `wlan.addr == [your router BSSID]` — Shows all frames involving your specific router.

What to Explore

Click on one of the four EAPOL frames and expand the packet tree in the detail panel. You will see the actual cryptographic material exchanged during the WPA2 handshake — the ANonce, SNonce, and MIC fields. This is the material that, in an authorized assessment context, gets fed into Hashcat for offline password evaluation. Seeing it in raw form is a powerful reminder of why strong Wi-Fi passwords matter.



Tip: Save this PCAP file. It is a record of your own network's handshake and makes excellent reference material as you continue learning. Label it clearly with the date and your router model so you can find it later.

Exercise 8: Deauth Detection — Watching for Attacks on Your Own Network

Goal: Use Deauth Sniff to monitor your own network for deauthentication frames and understand what normal versus suspicious traffic looks like.

Steps

- Step 1 — Start Deauth Sniff on your Marauder device near your router.
- Step 2 — Leave it running for 5 minutes during normal household Wi-Fi use.
- Step 3 — Note any deauth frames that appear. A small number is normal — devices legitimately disconnect and reconnect regularly.
- Step 4 — Now ask a housemate to intentionally disconnect their phone from Wi-Fi and reconnect. Watch the deauth frames appear for that event.
- Step 5 — Compare the normal background rate with the burst caused by a deliberate disconnection.

Understanding Normal vs Suspicious

A handful of deauth frames over several minutes is entirely normal — it reflects devices naturally managing their connections. What looks suspicious is a rapid, sustained burst of deauth frames targeting a specific BSSID or using the broadcast MAC address (FF:FF:FF:FF:FF:FF), which would indicate an attempt to disconnect all devices on a network simultaneously.

Running this exercise gives you a personal baseline for what normal deauth activity on your own network looks like — which makes you much better equipped to recognise abnormal activity in the future.

 **Tip:** If you want to simulate an attack for comparison, repeat Exercise 5 (deauth your own device) while Deauth Sniff is running in a second session or on a second device. You will immediately see the visual difference between one or two normal deauth frames and a burst of forged ones.

Putting It All Together — A Full Homelab Session

Once you are comfortable with the individual exercises, try running them as a single connected session. This mirrors how a real wireless security assessment flows from initial reconnaissance through to capture and analysis.

- Start with AP Scan — establish your full network map and note all BSSIDs and channels.
- Follow with Station Scan — identify every connected device and map it to a physical object.
- Run Probe Sniff — capture what your devices are broadcasting about their network history.
- Enable Save PCAP and run Beacon Sniff — capture a minute of beacon traffic.
- Trigger a WPA2 handshake by disconnecting and reconnecting a device.
- Run Deauth Sniff briefly to observe normal management frame activity.
- Export all PCAPs and open them in Wireshark — explore each capture with the filters from Exercise 7.

By the end of this full session you will have performed, on your own equipment, the core passive reconnaissance and capture workflow that underpins real-world Wi-Fi security analysis. Everything from there is refinement and depth — the foundations are solid.

Post-Capture Workflow: From Raw Data to Actionable Analysis

Capturing data with Marauder is only the first half of the job. The second half — transferring, verifying, and analysing what you collected — is where the real learning happens and where the value of each session is realised. This chapter pulls the entire post-capture process together into one cohesive workflow, covering every tool and step from the moment you stop a capture to the moment you have meaningful results.

Step 1: Stop Cleanly and Verify On-Device

Before removing the SD card or powering off, always stop the active scan or capture explicitly using the Marauder menu or the stopscan CLI command. This allows the firmware to close open file handles and flush the write buffer to the SD card properly. Removing the card while a write is in progress is the most common cause of corrupted or truncated PCAP files.

- Stop the active scan from the menu or via stopscan in the CLI
- Navigate to the SD Card Browser and confirm your capture files exist and have a non-zero file size
- If a file shows 0 bytes, the capture failed — check the troubleshooting section before proceeding

 **Tip:** A quick file size check in the SD Card Browser takes ten seconds and saves you from discovering a failed capture only after you have transferred everything to your computer.

Step 2: Transfer Files to Your Computer

Power off the device, remove the SD card, and insert it into your computer using a card reader. The SD card will appear as a removable drive. Navigate to the relevant directories and copy your files:

- /pcaps/ — PCAP capture files from AP Scan, Beacon Sniff, PMKID Sniff, and handshake captures
- /wardrive/ — Wardriving CSV logs with GPS-tagged network data
- /logs/ — General activity logs

Copy the files to a clearly labelled folder on your computer before doing anything else. Use a naming convention that includes the date and session type — for example: 2024-03-15_home_pmkid_capture.pcap. Organised storage makes it much easier to find specific captures later.

 **Tip:** Create a dedicated folder structure for your Marauder sessions: one parent folder per month, subfolders per session. PCAPs and wardriving CSVs accumulate quickly once you start capturing regularly.

Step 3: Verify PCAP Integrity in Wireshark

Open Wireshark and drag your PCAP file onto the window, or use File > Open. Wireshark will load the capture and display the frame list. The first thing to check is whether the file loaded without errors and contains frames.

If Wireshark Reports a Damaged File

Try opening it anyway by clicking OK on the error dialog — partial captures often contain usable frames before the truncation point. If the file is severely corrupted, try the pcapfix tool from the command line:

```
pip install pcapfix  
pcapfix your_capture.pcap
```

pcapfix attempts to rebuild a valid PCAP structure from damaged data and often recovers the majority of frames from truncated files.

Key File Health Checks in Wireshark

- **Frame count:** A completely empty PCAP with zero frames means the capture never started or failed immediately. Check that Save PCAP was enabled before scanning.
- **Duration:** The capture duration shown in the status bar at the bottom of Wireshark should roughly match how long you ran the scan. A five-minute scan should produce a capture of approximately five minutes duration.
- **Protocol distribution:** Go to Statistics > Protocol Hierarchy. You should see 802.11 (Wi-Fi) as the dominant protocol. If you see mostly Ethernet frames, the PCAP may have been captured from the wrong interface.

Step 4: Analyse PCAP Captures — Wireshark Filter Reference

Wireshark's filter bar is the primary tool for navigating a wireless PCAP. Type a filter expression into the bar at the top of the frame list and press Enter to apply it. Here is a practical reference of the most useful filters for Marauder captures:

Authentication & Handshake Filters

- `eapol` — Shows all WPA2/WPA3 four-way handshake frames. This is the first filter to apply after a PMKID or handshake capture. A complete capture shows exactly four EAPOL frames per handshake.
- `eapol && wlan.addr == [BSSID]` — Narrow EAPOL results to a specific access point. Replace [BSSID] with your router's MAC address, e.g. `eapol && wlan.addr == aa:bb:cc:dd:ee:ff`
- `wlan_rsn_eapol.keydes.msgnr == 1` — Shows only Message 1 of the handshake (ANonce from AP). Useful for confirming PMKID availability.
- `wlan_rsn_eapol.keydes.msgnr == 2` — Shows only Message 2 (SNonce + MIC from client). This frame contains the material used in offline password auditing.

Network Discovery Filters

- `wlan.fc.type_subtype == 8` — Beacon frames only. Shows all APs that were broadcasting during the capture.
- `wlan.fc.type_subtype == 4` — Probe request frames. Shows client devices searching for remembered networks and the SSIDs they are looking for.
- `wlan.fc.type_subtype == 5` — Probe response frames. Shows APs responding to probe requests.
- `wlan.ssid == "YourNetworkName"` — Filter all frames related to a specific SSID. Replace with your network name in quotes.
- `wlan.bssid == aa:bb:cc:dd:ee:ff` — Filter all frames involving a specific access point MAC address.

Attack & Management Frame Filters

- `wlan.fc.type_subtype == 12` — Deauthentication frames. Shows all deauth activity in the capture — both legitimate disconnections and potential attack traffic.
- `wlan.fc.type_subtype == 10` — Disassociation frames. Similar to deauth — used to end client-AP associations.
- `wlan.fc.type_subtype == 12 && wlan.da == ff:ff:ff:ff:ff:ff` — Broadcast deauth frames. If you see a flood of these, a deauth attack targeting all clients simultaneously was active during capture.

Signal & Device Filters

- `wlan.ta == aa:bb:cc:dd:ee:ff` — Frames transmitted by a specific device (transmitter address).
- `wlan.ra == aa:bb:cc:dd:ee:ff` — Frames received by a specific device (receiver address).
- `radiotap.dbm_antsignal > -60` — Frames received with signal stronger than -60 dBm. Useful for isolating nearby devices from distant background noise.

Step 5: Extract the Handshake for Password Auditing (Authorized Only)

If your capture contains a complete four-way handshake or PMKID from your own network and you want to audit your password strength, the next step is converting the PCAP to a format Hashcat can process.

⚠ This step must only be performed on credential material captured from networks you own or are explicitly authorized to test.

Install hcxtools

```
pip install hcxtools # or via package manager on Linux
```

Convert PCAP to Hashcat Format

```
hcxpcapngtool -o capture.hc22000 your_capture.pcap
```

The output file `capture.hc22000` contains the extracted hash in WPA-PBKDF2-PMKID+EAPOL format (Hashcat mode 22000), which handles both PMKID and full handshake material in a single format.

Verify the Hash Was Extracted

```
cat capture.hc22000
```

If the file is not empty, the extraction succeeded. An empty output file means the PCAP did not contain a usable PMKID or complete handshake — return to Marauder and capture again, ensuring a client device connects or reconnects during the capture window.

Run Hashcat Against a Wordlist

```
hashcat -m 22000 capture.hc22000 wordlist.txt
```

The RockYou wordlist (rockyou.txt) is the standard first test — it contains over 14 million commonly used passwords and is freely available online. If your password appears in it, change it immediately.

For a more thorough test, use Hashcat's rule-based mutation mode, which transforms wordlist entries with common substitutions (@ for a, 3 for e, appended numbers, capitalisation variations):

```
hashcat -m 22000 capture.hc22000 wordlist.txt -r rules/best64.rule
```

 **Tip:** A password that survives both a RockYou test and a best64 rules test is meaningfully stronger than most home Wi-Fi passwords. A truly secure password is 16+ random characters — something that would not appear in any wordlist and cannot be derived from a simple mutation of a common word.

Step 6: Analyse Wardriving Data

Wardriving CSV files from Marauder contain a structured record of every network detected during your session, with GPS coordinates attached to each entry. The post-capture workflow for wardriving data is separate from PCAP analysis.

Verify the CSV Before Uploading

Open the CSV in a spreadsheet application (Excel, LibreOffice Calc, or Google Sheets). Check:

- GPS columns (latitude, longitude) contain real coordinate values — not zeros. Any rows with 0.000000 coordinates had no GPS fix at detection time.
- The SSID and BSSID columns are populated. Empty rows indicate a logging error.
- The timestamp column looks correct for your session date and time.

If a significant portion of rows have 0.0 coordinates, the GPS fix was not established at the start of the session. Only rows with valid coordinates are useful for mapping and WiGLE upload.

Visualise Locally with GPS Visualizer

- Step 1 — Go to gpsvisualizer.com and click 'Convert & Map GPS Data'.
- Step 2 — Upload your wardriving CSV.
- Step 3 — Set the output format to Google Maps or Leaflet (browser-based map).
- Step 4 — Click 'Map It'. Your route appears as a coloured track on the map with detected networks plotted as points.

Upload to WiGLE

- Step 1 — Log in to wiggles.net.
- Step 2 — Click 'Upload' in the top navigation.
- Step 3 — Select your CSV file and click Upload.
- Step 4 — Wait for processing (a few minutes for large files).
- Step 5 — Check your profile page to see updated statistics and the new networks on your personal map.

Step 7: Document Your Session

Consistent session documentation pays dividends over time — especially as you accumulate months of captures and wardriving logs. A simple text note alongside each session folder takes two minutes to write and makes every future reference to that data much faster.

A minimal session note should include: the date and time, what hardware was used, what functions were run, what the goal was, what was found (or not found), and any anomalies noticed. Over time, these notes become a personal log of your learning and a reference for what worked and what did not.

💡 Tip: Even a single sentence per session is better than nothing. 'Home PMKID capture, full handshake obtained, password survived RockYou test, changed password to 20-char random passphrase.' That is enough context to make the capture file meaningful six months later.

Ethical Scenario Walkthroughs

These step-by-step walkthroughs cover complete authorized use cases from start to finish. Each scenario is designed to be performed only on your own equipment and networks, or on networks you are explicitly authorized to test.

Scenario 1: Auditing Your Own Home Wi-Fi Password Strength

Goal: Determine whether your home Wi-Fi password could be guessed by an attacker using offline analysis tools. This is one of the most practically valuable things you can do with Marauder on your own network.

What You Will Need

- Marauder device with SD card
- A computer running Wireshark (free download at [wireshark.org](https://www.wireshark.org))
- Optional: Hashcat (for actual password auditing — advanced)

Step-by-Step

- Step 1: Insert a formatted SD card and power on your Marauder device.
- Step 2: Run AP Scan and identify your home network in the list. Note its BSSID and channel.
- Step 3: Enable Save PCAP in Marauder settings.
- Step 4: Run PMKID Sniff and wait for your router to expose a PMKID. This typically takes under a minute near your router.
- Step 5: Stop the capture and navigate to SD Card Browser to confirm the PCAP was saved.
- Step 6: Transfer the PCAP file to your computer.

- Step 7: Open the file in Wireshark and filter by 'eapol' to inspect the captured authentication material.
- Step 8 (Optional / Advanced): Use Hashcat with a dictionary file to test whether your password appears in a common password list. If it does, change your Wi-Fi password immediately.

⚠ Never perform this on a network you do not own. Even if you live in an apartment building and your neighbor's network appears stronger, testing it without permission is illegal.

💡 Tip: If your password appears in a Hashcat dictionary test, switch to a password of at least 16 characters mixing upper/lowercase letters, numbers, and symbols. A long random passphrase is ideal.

Scenario 2: Mapping Your Home Network with AP Scan + Station Scan

Goal: Get a complete picture of every device connected to your home network and identify anything unexpected.

Step-by-Step

- Step 1: Power on Marauder and run AP Scan. Wait for your home network(s) to appear in the list.
- Step 2: Note the BSSID(s) of your router(s). If you have a mesh network, multiple BSSIDs may appear with the same SSID.
- Step 3: Run Station Scan. Allow 30-60 seconds for client devices to appear.
- Step 4: Review the client list. For each device, note the MAC address.
- Step 5: Cross-reference MAC addresses with known devices. The first three bytes (OUI) identify the manufacturer — look them up at macvendors.com to identify unknown devices.
- Step 6: Log into your router's admin panel and compare Marauder's client list with the router's DHCP table to reconcile any discrepancies.

💡 Tip: MAC address randomization on modern phones means the MAC you see in Marauder may not match what your router records. Disable MAC randomization temporarily on your own devices to get accurate readings during an audit.

Scenario 3: Checking Your Area for Deauth Attacks

Goal: Passively monitor your wireless environment for signs of deauthentication attacks — a useful defensive check if you suspect Wi-Fi interference or disruption.

Step-by-Step

- Step 1: Power on Marauder in the area you want to monitor.
- Step 2: Run AP Scan first to establish a baseline of local networks.
- Step 3: Start Deauth Sniff. Marauder will now listen for deauthentication frames in the environment.

- Step 4: Allow the scan to run for several minutes. Normal environments will show occasional legitimate deauth frames (devices disconnecting normally).
- Step 5: A sudden flood of deauth frames targeting a specific BSSID or broadcast address indicates a possible deauth attack in progress.
- Step 6: If you detect a suspicious pattern on your own network, note the time and investigate your router logs. You may also want to enable Protected Management Frames (PMF / 802.11w) on your router if supported.

💡 Tip: Most modern routers support 802.11w (Protected Management Frames), which makes deauthentication frames cryptographically verified and thus much harder to spoof. Check your router settings and enable it if available.

Scenario 4: Your First Wardriving Session

Goal: Complete a successful wardriving session from setup to WiGLE upload.

Before You Leave the House

- Charge your Marauder device fully.
- Insert a formatted SD card with adequate free space (1GB free is plenty for a typical session).
- Power on the device indoors and verify the SD card is detected.
- Check that firmware is up to date.

Getting Your GPS Fix

- Step 1: Go outdoors or position your device near a window with clear sky view.
- Step 2: Open GPS Monitor. Watch the satellite count climb — aim for at least 4 satellites before starting.
- Step 3: Verify coordinates look correct for your location (cross-check with your phone's Maps app).

Running the Session

- Step 4: Start Wi-Fi Wardrive Logging.
- Step 5: Optionally start Bluetooth Wardrive Logging as well.
- Step 6: Travel your chosen route. You do not need to do anything else — Marauder logs automatically.
- Step 7: When finished, stop the logging and return home.

After the Session

- Step 8: Power off and remove the SD card.
- Step 9: Insert the card into your computer and navigate to the /wardrive/ folder.
- Step 10: Open the CSV log in a spreadsheet application and check that GPS coordinates are populated correctly.

- Step 11: Log in to WiGLE and upload the CSV.
- Step 12: Check your WiGLE profile after processing to see your new networks and updated stats.

💡 Tip: Your first session does not need to cover a large area. Even a 15-minute walk around your neighborhood is a valid wardriving run and a great way to learn the workflow end-to-end before attempting longer sessions.

Scenario 5: Probe Request Privacy Demonstration

Goal: Demonstrate to a friend or family member how their device broadcasts previously connected network names — a real-world privacy awareness exercise using only your own devices.

Step-by-Step

- Step 1: Power on Marauder and start Probe Sniff.
- Step 2: Ask your volunteer to take their phone, turn Wi-Fi off, wait 10 seconds, then turn Wi-Fi back on.
- Step 3: Watch Marauder — within seconds you will likely see probe requests from their device broadcasting the names of networks it has previously connected to.
- Step 4: Show them the results. Explain that every network name in that list is a location their device remembers: home, work, hotels, coffee shops, friends' houses.
- Step 5: Discuss the implications: an attacker with a device like Marauder could passively collect this data in public spaces without anyone knowing.
- Step 6: Show them how to reduce exposure — on most phones, clearing saved Wi-Fi networks and enabling MAC address randomization significantly reduces probe request leakage.

💡 Tip: This demonstration works best with older phones or phones that have MAC randomization disabled. Modern iOS and Android devices randomize their MAC and increasingly suppress specific SSIDs in probe requests — which is itself a good topic for discussion.

Glossary

Quick-reference definitions for terms used throughout this guide.

802.11: The IEEE standard that defines Wi-Fi protocols. Different suffixes (a/b/g/n/ac/ax) indicate generations with different speeds and frequencies.

AP (Access Point): A network device that creates a wireless local area network. Usually a router or base station. Identified by its BSSID.

Beacon Frame: A management frame broadcast by an AP roughly every 100ms to advertise its presence, SSID, capabilities, and supported data rates.

BLE (Bluetooth Low Energy): A low-power variant of Bluetooth designed for devices that need long battery life. Used in fitness trackers, sensors, beacons, and asset trackers.

BSSID (Basic Service Set Identifier): The hardware MAC address of a specific access point radio. Used to uniquely identify an AP even if its SSID is hidden or shared with another network.

Channel: A specific frequency range within a Wi-Fi band on which a network operates. In the 2.4 GHz band, common non-overlapping channels are 1, 6, and 11.

CLI (Command Line Interface): A text-based interface for controlling software. Marauder's serial CLI allows scripted or automated control via UART commands.

dBm (Decibels relative to one milliwatt): The unit used to measure wireless signal strength (RSSI). Always expressed as a negative number — closer to zero means stronger signal.

Deauthentication Frame: A Wi-Fi management frame used to formally terminate a client-AP session. Historically spoofable due to lack of frame authentication in older 802.11 standards.

ESP32: A low-cost, Wi-Fi and Bluetooth-capable microcontroller made by Espressif Systems. The hardware foundation of the Marauder platform.

ESPTool: Espressif's official command-line utility for flashing firmware to ESP32 and related chips.

FAT32: A file system format required by Marauder for SD cards. Supports cards up to 32GB natively. Must be used instead of exFAT or NTFS.

Firmware: The software installed on a hardware device that controls its behavior. ESP32 Marauder firmware is developed and maintained by JustCallMeKoko.

GPS (Global Positioning System): A satellite-based navigation system. Marauder uses an external GPS module to attach geographic coordinates to wardriving log entries.

Hashcat: An open-source password recovery and auditing tool. Used in authorized security testing to evaluate the guessability of captured Wi-Fi credentials.

ILI9341: A common TFT display driver chip used in many DIY Marauder builds. Typically found in 2.8" and 3.2" color displays.

MAC Address (Media Access Control Address): A unique hardware identifier assigned to a network interface. The first three bytes (OUI) identify the manufacturer.

NEO-6M / NEO-7M / NEO-8M: Popular u-blox GPS module families used in DIY Marauder builds. Connect via UART and provide NMEA sentence output for location data.

NFC (Near Field Communication): Short-range wireless technology used for contactless payments and access cards. Supported by Flipper Zero but not natively by ESP32 Marauder.

OUI (Organizationally Unique Identifier): The first three bytes of a MAC address that identify the network interface manufacturer. Used to identify unknown devices during network audits.

PCAP (Packet Capture): A standardized file format for storing raw captured network frames. Readable by tools like Wireshark for detailed offline analysis.

PMKID (Pairwise Master Key Identifier): A cryptographic value included in WPA2 handshake messages that is derived from the network password. Can be captured and used for offline credential strength testing in authorized environments.

PMF (Protected Management Frames / 802.11w): A Wi-Fi security extension that cryptographically authenticates management frames (including deauth frames), making them significantly harder to spoof.

Probe Request: A management frame sent by a Wi-Fi client device when scanning for previously connected networks. Contains the SSIDs of remembered networks — a source of device history leakage.

RSSI (Received Signal Strength Indicator): A measurement of how strong a received wireless signal is. Expressed in dBm. Values closer to 0 indicate stronger signals.

SSID (Service Set Identifier): The human-readable name of a Wi-Fi network as it appears in device network lists.

Station: In Wi-Fi terminology, a client device (phone, laptop, IoT device) that is associated with an access point.

UART (Universal Asynchronous Receiver-Transmitter): A serial communication protocol used to connect the Marauder device to a GPS module or a computer's serial console.

Wardriving: The practice of traveling through physical space while scanning and logging wireless networks with GPS coordinates. A long-standing community hobby with a global mapping community.

WiGLE (Wireless Geographic Logging Engine): The largest community-driven wireless mapping platform. Accepts wardriving log uploads and maintains a global database of mapped Wi-Fi and Bluetooth networks.

Wireshark: A free, open-source network protocol analyzer. The standard tool for opening and analyzing PCAP files.

WPA2 / WPA3: Wi-Fi Protected Access versions 2 and 3. The current standard security protocols for wireless networks. WPA3 adds stronger encryption and resistance to offline password guessing.

Quickstart Checklist: First 10 Steps

Just got your Marauder device and not sure where to begin? Follow these steps in order. This checklist assumes you have a prebuilt or freshly assembled device — adjust steps as needed for your setup.

Before Powering On

- Step 1 — Prepare your SD card. Format a card (32GB or smaller) as FAT32 and insert it into the device before first boot.
- Step 2 — Use a data-capable USB cable. Plug the device into your computer. Confirm it is detected (a COM port appears in Device Manager on Windows, or in `/dev/` on Linux/macOS).
- Step 3 — Identify your hardware variant. Check the board name, display type, and chip variant. Write it down — you will need it for firmware selection.
- Step 4 — Download the correct firmware. Visit the official JustCallMeKoko GitHub releases page and download the .bin file that exactly matches your hardware. When in doubt, ask the community.

Flashing & First Boot

- Step 5 — Flash the firmware. Use the web flasher (Chrome/Edge) or ESPTool. Select your exact device type. Do not disconnect during flashing.
- Step 6 — First boot. Disconnect and reconnect the device after flashing. The screen should display the Marauder interface. If the screen is blank, see the Troubleshooting section.
- Step 7 — Verify SD card detection. Navigate to the SD Card Browser in the menu. If the card is detected, you will see the directory structure. If not, see the SD Card Troubleshooting section.

Your First Functions

- Step 8 — Run your first AP Scan. From the main menu, start an AP Scan. Within seconds you should see nearby Wi-Fi networks appearing. If nothing appears, check that Wi-Fi scanning is enabled in settings.
- Step 9 — Run a Station Scan. Follow up the AP Scan with a Station Scan to see connected client devices.

- Step 10 — Save a PCAP. Enable Save PCAP in settings, run a short AP or Beacon Sniff, then check the SD Card Browser to confirm a .pcap file was created. You are now capturing wireless data.

💡 Tip: Bookmark the official [GitHub page](#) and the community [Discord](#). Marauder updates frequently and the community is your best resource for firmware news, hardware questions, and troubleshooting help.

Pinout & Wiring Reference

This section provides general wiring guidance for the most common DIY Marauder component combinations. Always cross-reference with the specific pinout documentation for your firmware build and board variant — pin assignments can vary between boards and firmware versions.

⚠️ **Pin assignments are not universal. A pinout that works for one ESP32 board may not work for another. Always verify against the official documentation for your specific hardware and firmware combination before wiring.**

ILI9341 TFT Display (SPI)

The ILI9341 display connects via SPI. Typical connections on a standard ESP32 DevKit:

- VCC → 3.3V
- GND → GND
- CS → GPIO 5
- RESET → GPIO 4
- DC (Data/Command) → GPIO 2
- MOSI (SDI) → GPIO 23
- SCK (Clock) → GPIO 18
- LED (Backlight) → 3.3V or PWM pin for brightness control

💡 Tip: Some ILI9341 breakout boards also have a MISO pin. For display-only use, MISO is not required — leave it unconnected unless your build also uses the SPI bus for SD card.

SD Card Module (SPI)

SD card modules also use SPI and can share the SPI bus with the display, using separate CS (Chip Select) pins:

- VCC → 3.3V (use a module with onboard 3.3V regulator if powering from 5V)
- GND → GND
- CS → GPIO 13 (separate CS from display)
- MOSI → GPIO 23 (shared with display)
- SCK → GPIO 18 (shared with display)
- MISO → GPIO 19

💡 Tip: Always use a 3.3V-compatible SD card module with the ESP32. Modules designed for 5V Arduino use may damage the ESP32's GPIO pins unless they include level-shifting circuitry.

GPS Module (UART)

GPS modules like the NEO-6M communicate via UART (serial). They use a separate pair of pins from the SPI bus:

- VCC → 3.3V or 5V (check your module's datasheet — NEO-6M can accept both)
- GND → GND
- TX (GPS) → GPIO 16 (ESP32 RX2)
- RX (GPS) → GPIO 17 (ESP32 TX2)

Note: GPS module TX connects to ESP32 RX, and GPS RX connects to ESP32 TX. This is standard UART crossover wiring.

💡 Tip: The default baud rate for most NEO-6M modules is 9600. Verify this matches your firmware's GPS baud rate configuration. A mismatch will result in garbled or absent GPS data.

Touch Screen (XPT2046, if applicable)

Some ILI9341 display modules include a resistive touchscreen controlled by an XPT2046 chip. If your build uses touch input:

- T_CS → GPIO 14
- T_IRQ → GPIO 27 (optional interrupt pin)
- T_CLK, T_DIN, T_DO → share SPI bus (GPIO 18, 23, 19)

Full Build Wiring Summary

A complete DIY build with display, SD card, and GPS typically uses the following GPIO pins simultaneously. Confirm there are no conflicts with your specific firmware version:

- GPIO 2, 4, 5, 18, 23 — Display (ILI9341 SPI)
- GPIO 13, 18, 19, 23 — SD Card (SPI, shared bus)
- GPIO 16, 17 — GPS Module (UART2)
- GPIO 14, 27 — Touchscreen (if equipped)

⚠️ **Always power off the ESP32 before connecting or disconnecting components. Connecting or disconnecting pins while powered can permanently damage GPIO pins or connected modules.**

Serial Console Command Reference

Marauder's CLI (Command Line Interface) allows you to control the device via text commands over a USB serial connection. This is useful for automation, scripting, headless operation, and accessing functions not exposed in the on-screen menu.

Connecting to the Serial Console

- Baud rate: 115200
- Windows: Use PuTTY, Tera Term, or the Arduino IDE Serial Monitor
- macOS/Linux: Use 'screen /dev/ttyUSB0 115200' or minicom
- Browser: Some web-based serial terminals work in Chrome/Edge with Web Serial API

💡 Tip: In the Arduino IDE, set the baud rate to 115200 and select 'Both NL & CR' as the line ending for commands to be processed correctly.

Core Commands

Scanning

- scanap — Start an AP scan
- scanst — Start a station scan
- sniffbeacon — Start beacon sniffing
- sniffdeauth — Start deauthentication frame sniffing
- sniffpmkid — Start PMKID sniffing
- sniffprobe — Start probe request sniffing
- stopscan — Stop any active scan or sniff

Wardriving

- wardrive — Start Wi-Fi wardriving log
- wdstop — Stop wardriving

PCAP & Logging

- savepcap on — Enable PCAP saving to SD card
- savepcap off — Disable PCAP saving
- clearpcap — Delete all saved PCAP files

GPS

- gpsinfo — Display current GPS fix status, coordinates, and satellite count

System

- info — Display firmware version, build date, and hardware target
- reboot — Restart the device

- `cleard` — Clear all files from SD card (use with caution)
- `ls` — List files on the SD card

Attack / Active Functions (Authorized Use Only)

- `attack -t deauth -b [BSSID]` — Deauth attack against a specific AP (authorized testing only)
- `attack -t beacon -s [SSID]` — Beacon spam with specified SSID
- `attack -t probe` — Probe request flood
- `stopattack` — Stop any active attack function

⚠ Active attack commands must only be used on your own equipment or networks you are explicitly authorized to test. Unauthorized use of these commands is illegal.

 **Tip:** CLI commands and their syntax can change between firmware versions. Run `'help'` to see a current list of available commands on your installed firmware version.

Known Limitations & Hardware Quirks

Understanding what Marauder cannot do is just as important as knowing what it can. Many common beginner questions stem from expecting capabilities the ESP32 hardware simply does not support. This section sets realistic expectations.

Wi-Fi Limitations

No 5 GHz Support

The ESP32's built-in Wi-Fi radio operates exclusively on the 2.4 GHz band. It cannot scan, capture, or interact with 5 GHz Wi-Fi networks at all. This is a hardware limitation, not a firmware one — no firmware update will ever add 5 GHz support to standard ESP32 hardware.

 **Tip:** If you need 5 GHz monitoring, that requires dedicated hardware such as a laptop with a monitor-mode capable Wi-Fi card running tools like Aircrack-ng or Kismet.

No Simultaneous Multi-Channel Scanning

The ESP32 has a single Wi-Fi radio and must hop between channels sequentially. It cannot monitor all channels simultaneously. This means networks that only appear briefly on a channel Marauder is not currently hopping to may be missed. This is normal behavior, not a malfunction.

WPA3 Handshake Capture Limitations

WPA3's Simultaneous Authentication of Equals (SAE) protocol significantly reduces the effectiveness of traditional PMKID and four-way handshake capture. Marauder's PMKID sniff may not yield usable material from WPA3 networks. This is by design — WPA3 was specifically engineered to resist this type of offline credential auditing.

No Full Packet Injection on All Platforms

While Marauder can transmit certain frame types (deauth, beacon, probe), it does not support full arbitrary packet injection in the way that dedicated wireless security tools on Linux do. The ESP32's wireless driver imposes constraints on what frames can be crafted and transmitted.

Bluetooth Limitations

BLE Only for Most Functions

Most Marauder Bluetooth functions target BLE (Bluetooth Low Energy). Classic Bluetooth (the kind used by speakers and headphones) has limited support. Do not expect to scan for or interact with classic Bluetooth audio devices in the way you can with BLE sensors and trackers.

BLE MAC Address Randomization

Modern iOS and Android devices use randomized BLE MAC addresses that change periodically. This means BLE scans may show the same physical device with different MAC addresses on different scans. It also means tracking a specific device over time by MAC address is increasingly unreliable on modern hardware.

Hardware Quirks

SD Card Write Speed Bottleneck

The ESP32's SPI interface limits SD card write speed. During heavy PCAP capture on busy networks, the write queue can back up, occasionally causing dropped frames. This is normal on the ESP32 platform. Use a fast SD card (Class 10 / UHS-I) to minimize this effect.

GPS Cold Start Time

The NEO-6M and similar modules can take 1-5 minutes to acquire a GPS fix from cold start (no cached almanac data). Once almanac data is cached, warm starts are much faster. If you need instant GPS fixes, leave the module powered for a few minutes before starting a wardriving session.

Battery Voltage Drop Under Load

Running Wi-Fi scanning, GPS, display backlight, and SD card writes simultaneously puts significant current draw on the battery. This can cause voltage sag that triggers the ESP32's brownout detector, causing unexpected resets. If you experience frequent resets during heavy use, check your battery's discharge curve and consider a larger capacity battery or a lower backlight brightness setting.

Wi-Fi MAC Address Randomization on Modern Clients

Modern smartphones use randomized MAC addresses when scanning for networks and when connected. This means Station Scan and Probe Sniff results may show changing MAC addresses from the same physical device across sessions. This is normal — it is a privacy protection built into iOS 14+, Android 10+, and Windows 10+.

Recommended Companion Tools & Software

Marauder is most powerful when used alongside the right companion tools. Here is a curated list of free software that extends and complements what Marauder can do.

Packet Analysis

Wireshark

The industry-standard free packet analysis tool. Open Marauder's PCAP files and inspect every captured frame in detail. Filter by protocol, follow conversations, decode encryption negotiation, and visualize traffic patterns. Essential for anyone serious about wireless analysis.

- Platform: Windows, macOS, Linux
- Cost: Free and open source
- Get it: [wireshark.org](https://www.wireshark.org)

hcxtools

A set of command-line tools specifically designed for converting and working with Wi-Fi handshake and PMKID capture files. Converts PCAP files to formats usable by Hashcat for authorized credential auditing.

- Platform: Linux (primary), macOS (via Homebrew)
- Cost: Free and open source
- Get it: github.com/ZerBea/hcxtools

Password Auditing (Authorized Use Only)

Hashcat

The leading open-source password recovery tool. Used in authorized security testing to evaluate whether a captured Wi-Fi credential hash matches entries in a wordlist or can be found via rule-based mutation. GPU-accelerated for fast processing.

- Platform: Windows, macOS, Linux
- Cost: Free and open source
- Get it: hashcat.net

⚠ Use Hashcat only on credential material you captured from networks you own or are authorized to test. Using it on others' credentials is illegal.

GPS & Mapping

GPS Visualizer

A free web tool that accepts GPS data in various formats and renders it as an interactive map. Great for visualizing your wardriving routes, coverage areas, and network density. Upload your Marauder wardriving CSV and see your route on a satellite map.

- Platform: Web-based
- Cost: Free
- Get it: gpsvisualizer.com

QGIS

A free desktop GIS (Geographic Information System) application for advanced wardriving data visualization and analysis. Overkill for casual use, but powerful if you want to overlay wardriving data on custom maps, perform spatial analysis, or produce publication-quality coverage maps.

- Platform: Windows, macOS, Linux
- Cost: Free and open source
- Get it: qgis.org

Device & Network Identification

MAC Vendors (macvendors.com)

A web-based OUI lookup tool. Paste a MAC address and it identifies the network interface manufacturer. Invaluable during home network audits for identifying unknown devices found in Station Scan results.

Wigle.net

Beyond being the wardriving upload platform, WiGLE's web interface lets you look up any BSSID or SSID in the database to see where it has been detected geographically. Useful for verifying your own captures and exploring wireless density in any area.

Serial Communication

PuTTY (Windows)

A free terminal emulator that supports serial connections at any baud rate. Use it to connect to Marauder's CLI at 115200 baud for scripting and automation.

minicom (Linux/macOS)

The standard Linux serial terminal. Lightweight and scriptable. Available via package manager on most distributions (`apt install minicom`, `brew install minicom`).

Arduino IDE Serial Monitor

If you already have the Arduino IDE installed for firmware development, its built-in Serial Monitor works well for CLI interaction with Marauder. Set baud rate to 115200 and line ending to 'Both NL & CR'.

Changelog & Version Notes

Marauder firmware evolves rapidly. Use this section to track firmware versions you have used, what changed, and any hardware-specific notes. This is especially useful if you maintain multiple Marauder devices or share this guide with others.

How to Use This Section

When you update your firmware, add a new entry below with the version number, date, key changes from the release notes, and any personal observations about behavior changes on your specific hardware.

Version Log Template

Version: _____ Date Updated: _____
Hardware: _____ Firmware File Used: _____
Key changes: _____
Issues observed: _____
Notes: _____

 **Tip:** Copy the Version Log Template above for each new firmware update. Keeping a version history has saved many users from confusion when a feature disappears or changes behavior after an update.

Community Resources & Links

The Marauder community is active, welcoming, and a great resource for questions, hardware advice, and firmware news. Here are the most important places to find help and stay connected.

Official Resources

- **GitHub Repository:** github.com/justcallmekoko/ESP32Marauder — Source code, firmware releases, hardware documentation, and issue tracker. This is the authoritative source for everything Marauder.
- **GitHub Releases Page:** github.com/justcallmekoko/ESP32Marauder/releases — All firmware versions with changelogs and hardware-specific binaries. Always download firmware from here.

- **GitHub Wiki:** The official documentation wiki linked from the repository covers pinouts, hardware variants, and function references.

Community Spaces

- **Discord:** The primary real-time community hub. Search for the official ESP32 Marauder Discord — the GitHub README links to it. Great for hardware questions, build help, and firmware discussion.
- **Reddit:** r/esp32 and r/flipperzero are active communities where Marauder topics come up regularly. Useful for general ESP32 hardware questions and beginner advice.
- **YouTube:** Search 'ESP32 Marauder' for build guides, feature walkthroughs, and wardriving session videos. Community creators produce a steady stream of tutorial content.

Hardware Sourcing

- **Official Store:** Check JustCallMeKoko's GitHub profile and social channels for current official device availability.
- **AliExpress / Amazon:** For DIY components — ESP32 DevKit boards, ILI9341 displays, NEO-6M GPS modules, SD card modules, and LiPo batteries are all widely available.
- **Tindie:** A marketplace for small-run hardware where community vendors sometimes sell Marauder-compatible builds.

Wardriving Community

- **WiGLE:** wiggles.net — Global wardriving map and statistics platform. Create an account to upload your wardrive logs and join the community.
- **WiGLE Discord:** Active community of wardrivers. Good for route tips, leaderboard discussion, and mapping project collaboration.

Bug Reporting

If you find a bug in the Marauder firmware, report it on the GitHub Issues page with your firmware version, hardware details, and a clear description of how to reproduce the problem. The more detail you provide, the faster it can be investigated.

Frequently Asked Questions

Short answers to the questions that come up most often in the Marauder community.

Can Marauder crack Wi-Fi passwords?

No, not directly. Marauder can capture authentication material (PMKID, WPA2 handshakes) from networks you are authorized to test, which can then be analyzed offline with tools like

Hashcat. Whether a password can be 'cracked' from that material depends entirely on how strong the password is. A long, random password is practically uncrackable regardless of what tool you use.

Does it work on 5 GHz Wi-Fi?

No. The ESP32's built-in radio is 2.4 GHz only. 5 GHz is a hardware limitation that cannot be addressed by firmware updates. For 5 GHz monitoring you need dedicated hardware.

Is wardriving legal?

In most countries, passive wardriving — driving around and logging Wi-Fi network names and locations that are broadcast publicly — is legal. However, laws vary by country and jurisdiction. Some places have regulations around GPS data collection or wireless monitoring. Research the laws in your specific region before wardriving in public. When in doubt, consult a local legal resource.

Do I need a GPS module?

No. GPS is only required for wardriving (logging networks with geographic coordinates). All other functions — AP Scan, Station Scan, Probe Sniff, PMKID Sniff, PCAP capture, and everything else — work without GPS. A GPS module is a recommended addition for users interested in wardriving but it is entirely optional otherwise.

Will it work without a screen?

Yes. Marauder can operate in headless mode via the serial CLI over USB. You can run scans, capture PCAPs, and control all functions through text commands without any display connected. This is useful for automated or scripted workflows.

Can I use it to monitor my neighbor's Wi-Fi?

No. Monitoring, capturing, or interfering with networks you do not own or are not authorized to test is illegal in virtually every jurisdiction regardless of whether the signals pass through your property. Marauder is for use on your own networks and equipment, or in explicitly authorized security assessments.

How is Marauder different from Kali Linux / Aircrack-ng?

Kali Linux running on a laptop with a monitor-mode Wi-Fi adapter is significantly more powerful — faster processing, more tool options, 5 GHz support, and full packet injection. Marauder's advantages are portability, cost, and battery life. A Marauder device fits in a pocket, costs a fraction of a laptop, and can run for hours. They serve different use cases and many enthusiasts use both.

What is the difference between Marauder and a Pwnagotchi?

A Pwnagotchi is an AI-driven automated WPA handshake harvester that runs on a Raspberry Pi Zero. It passively collects handshakes and PMKIDs continuously as it encounters networks. Marauder is a broader multi-function wireless analysis platform controlled manually by the user. Marauder can actually detect Pwnagotchi devices via its Detect Pwnagotchi function.

My device showed up fine yesterday but today it won't boot — what happened?

Most likely causes: low battery causing brownout reset, a corrupt SD card preventing boot, or a firmware issue. Try removing the SD card and booting without it to isolate the cause. If the device boots without the SD card, the card (or a corrupted file on it) is the problem. If it still won't boot, try the recovery steps in the 'What To Do If Your Device Appears Bricked' section.

Can I add my own Evil Portal pages?

Yes. Create your HTML page, name it index.html (or as specified by your firmware version), copy it to the /portals/ directory on your SD card via a card reader, and select it from the Marauder Evil Portal menu. The firmware will serve it to any device that connects to the rogue AP. Remember: only use Evil Portal in authorized settings.

Where do I get help if I'm stuck?

In order of speed: (1) this guide's Troubleshooting section, (2) the official GitHub Wiki, (3) the GitHub Issues page for known bugs, (4) the community Discord for real-time help. When asking for help, always include your firmware version, hardware variant, and a description of exactly what you tried and what happened.

Quickstart Checklist: First 10 Steps

Just got your Marauder device and not sure where to begin? Follow these steps in order. This checklist assumes you have a prebuilt or freshly assembled device — adjust as needed for your specific setup.

Before Powering On

- Step 1 — Prepare your SD card. Format a card (32GB or smaller) as FAT32 and insert it into the device before first boot.
- Step 2 — Use a data-capable USB cable. Plug the device into your computer and confirm it is detected (a COM port appears in Device Manager on Windows, or in /dev/ on Linux/macOS).
- Step 3 — Identify your hardware variant. Note your board name, display type, and ESP32 chip variant. Write it down — you will need it for firmware selection.
- Step 4 — Download the correct firmware. Visit the official JustCallMeKoko GitHub releases page and download the .bin file that exactly matches your hardware. When in doubt, ask the community first.

Flashing & First Boot

- Step 5 — Flash the firmware. Use the web flasher (Chrome/Edge) or ESPTool. Select your exact device type. Do not disconnect during flashing.
- Step 6 — First boot. Disconnect and reconnect the device after flashing. The Marauder interface should appear on screen. If the screen is blank, see the Troubleshooting section.
- Step 7 — Verify SD card detection. Navigate to the SD Card Browser in the menu. If the card is detected you will see the directory structure. If not, check formatting and card compatibility.

Your First Functions

- Step 8 — Run your first AP Scan. From the main menu, start an AP Scan. Within seconds nearby Wi-Fi networks should appear. If nothing appears, verify Wi-Fi scanning is enabled in settings.
- Step 9 — Run a Station Scan. Follow up the AP Scan with a Station Scan to see connected client devices.
- Step 10 — Save a PCAP. Enable Save PCAP in settings, run a short Beacon Sniff, then check the SD Card Browser to confirm a .pcap file was created. You are now capturing wireless data.

 Tip: Bookmark the official [GitHub page](#) and the community [Discord](#). Marauder updates frequently and the community is your best resource for firmware news, hardware questions, and troubleshooting.

Pinout & Wiring Reference

This section provides general wiring guidance for the most common DIY Marauder component combinations. Always cross-reference with the specific pinout documentation for your firmware build and board variant — pin assignments can and do vary between boards and firmware versions.

⚠ Pin assignments are not universal. A pinout that works for one ESP32 board may not work for another. Always verify against the official documentation for your specific hardware and firmware combination before connecting anything.

ILI9341 TFT Display (SPI)

The ILI9341 display connects via SPI. Typical connections on a standard ESP32 DevKit:

- VCC → 3.3V
- GND → GND
- CS → GPIO 5
- RESET → GPIO 4
- DC (Data/Command) → GPIO 2
- MOSI (SDI) → GPIO 23

- SCK (Clock) → GPIO 18
- LED (Backlight) → 3.3V or a PWM pin for brightness control

💡 Tip: Some ILI9341 breakout boards also expose a MISO pin. For display-only use MISO is not required — leave it unconnected unless your build shares the SPI bus with an SD card module.

SD Card Module (SPI)

SD card modules also use SPI and can share the SPI bus with the display, using a separate CS pin per device:

- VCC → 3.3V (use a module with onboard 3.3V regulator if powering from 5V)
- GND → GND
- CS → GPIO 13 (separate CS from display)
- MOSI → GPIO 23 (shared with display on the SPI bus)
- SCK → GPIO 18 (shared with display)
- MISO → GPIO 19

💡 Tip: Always use a 3.3V-compatible SD card module with the ESP32. Modules designed for 5V Arduino may damage the ESP32's GPIO pins unless they include level-shifting circuitry.

GPS Module (UART)

GPS modules like the NEO-6M communicate via UART and use a separate pair of pins from the SPI bus:

- VCC → 3.3V or 5V (check your module's datasheet — NEO-6M tolerates both)
- GND → GND
- TX (GPS module output) → GPIO 16 (ESP32 RX2)
- RX (GPS module input) → GPIO 17 (ESP32 TX2)

GPS TX connects to ESP32 RX, and GPS RX connects to ESP32 TX. This standard UART crossover is the most common beginner wiring mistake.

💡 Tip: The default baud rate for most NEO-6M modules is 9600. Confirm this matches your firmware's GPS baud rate setting. A mismatch produces garbled or absent GPS data.

Touchscreen (XPT2046, if equipped)

Some ILI9341 display modules include a resistive touchscreen with an XPT2046 controller:

- T_CS → GPIO 14
- T_IRQ → GPIO 27 (optional interrupt pin)
- T_CLK, T_DIN, T_DO → share the SPI bus (GPIO 18, 23, 19)

Full Build GPIO Summary

A complete DIY build with display, SD card, and GPS typically uses these pins simultaneously:

- GPIO 2, 4, 5, 18, 23 — ILI9341 display (SPI)
- GPIO 13, 18, 19, 23 — SD card module (SPI shared bus)
- GPIO 16, 17 — GPS module (UART2)
- GPIO 14, 27 — Touchscreen controller (if equipped)

⚠ Always power off the ESP32 before connecting or disconnecting components. Connecting pins while powered can permanently damage GPIO pins or attached modules.

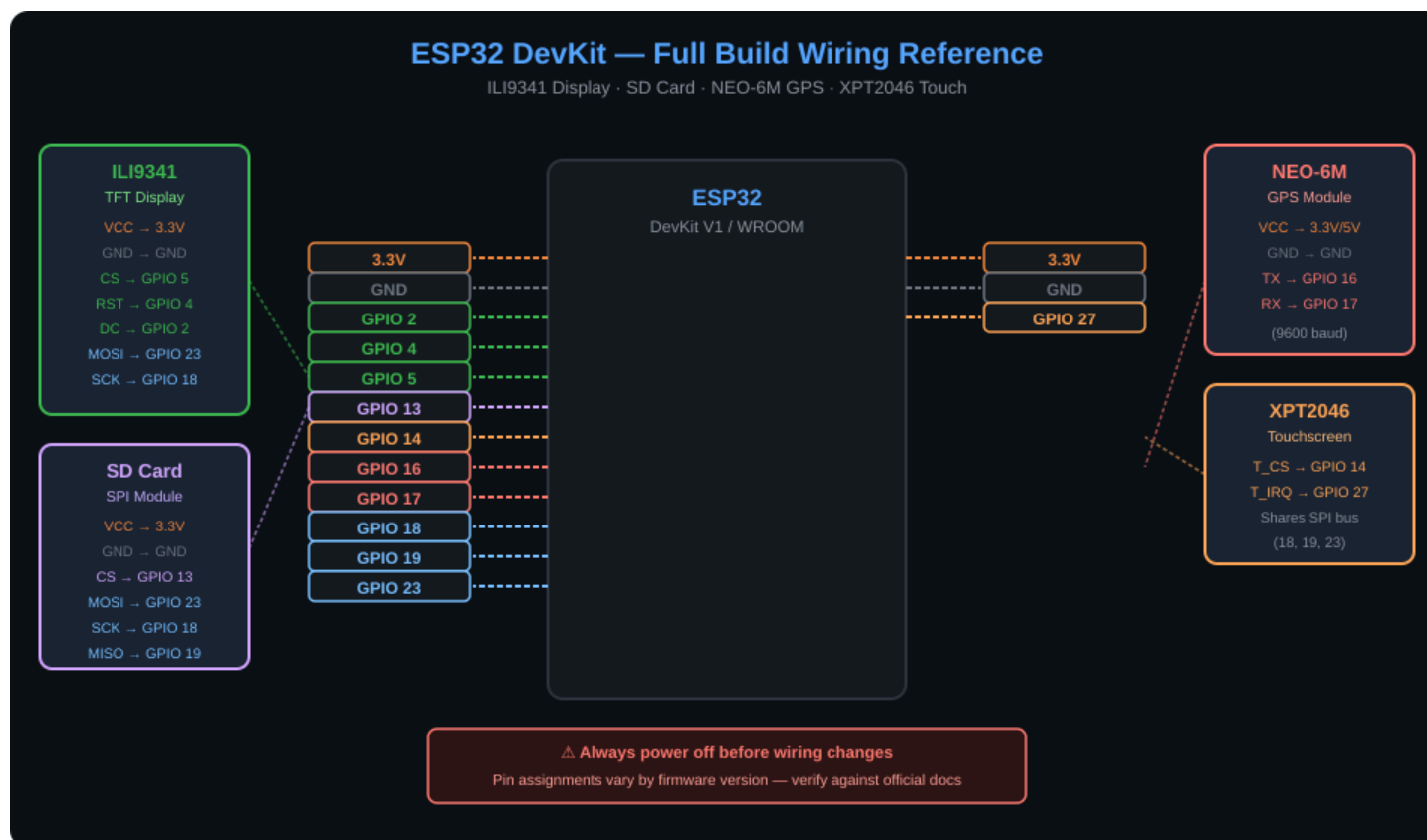


Figure 2 — ESP32 DevKit full build wiring: Display, SD Card, GPS, and Touchscreen

Serial Console Command Reference

Marauder's CLI allows you to control the device via text commands over a USB serial connection at 115200 baud. Useful for automation, scripting, headless operation, and functions not easily reached through the on-screen menu.

Connecting to the Console

- Windows: PuTTY, Tera Term, or Arduino IDE Serial Monitor — 115200 baud
- macOS / Linux: `screen /dev/ttyUSB0 115200` or `minicom`
- Browser: Web Serial terminals in Chrome/Edge

💡 Tip: In the Arduino IDE, set baud rate to 115200 and line ending to 'Both NL & CR' for commands to be processed correctly.

Scanning

- `scanap` — Start an AP scan
- `scanst` — Start a station (client) scan
- `sniffbeacon` — Start beacon frame sniffing
- `sniffdeauth` — Start deauth frame detection
- `sniffpmkid` — Start PMKID sniffing
- `sniffprobe` — Start probe request sniffing
- `stopscan` — Stop any active scan or sniff

Wardriving

- `wardrive` — Start Wi-Fi wardriving log (requires GPS fix)
- `wdstop` — Stop wardriving and close the log file

PCAP & Logging

- `savepcap on` — Enable PCAP saving to SD card
- `savepcap off` — Disable PCAP saving
- `clearpcap` — Delete all saved PCAP files

GPS & System

- `gpsinfo` — Display GPS fix status, coordinates, and satellite count
- `info` — Display firmware version, build date, and hardware target
- `reboot` — Restart the device
- `clearsd` — Clear all files from SD card (irreversible)
- `ls` — List files on the SD card
- `help` — List all available commands for your firmware version

Active / Attack Commands (Authorized Use Only)

- `attack -t deauth -b [BSSID]` — Deauth frames targeting a specific AP
- `attack -t beacon -s [SSID]` — Beacon spam with a specified SSID
- `attack -t probe` — Probe request flood

- stopattack — Stop any active attack function

⚠ **Active commands must only be used on networks and devices you own or are explicitly authorized to test. Always run 'help' to confirm syntax for your installed firmware version — commands can change between releases.**

Known Limitations & Hardware Quirks

Understanding what Marauder cannot do is just as important as knowing what it can. Many beginner frustrations stem from expecting capabilities the ESP32 hardware simply does not support.

Wi-Fi Limitations

No 5 GHz Support

The ESP32 radio operates exclusively on 2.4 GHz. It cannot scan, capture, or interact with 5 GHz networks under any circumstances. This is a permanent hardware constraint — no firmware update will ever add 5 GHz support.

💡 **Tip:** For 5 GHz monitoring you need a laptop with a monitor-mode capable Wi-Fi adapter running Kismet or Aircrack-ng on Linux.

No Simultaneous Multi-Channel Scanning

The ESP32 has one radio and must hop between channels sequentially. Networks that appear briefly on a channel Marauder is not currently tuned to may be missed. This is expected behavior.

WPA3 Handshake Limitations

WPA3's Simultaneous Authentication of Equals (SAE) protocol significantly reduces the effectiveness of PMKID capture. Marauder may yield little usable material from WPA3 networks — this is by design, as WPA3 was specifically engineered to resist offline credential auditing.

Limited Packet Injection

Marauder can transmit certain frame types (deauth, beacon, probe) but does not support full arbitrary packet injection the way dedicated Linux wireless tools do. The ESP32 wireless driver imposes constraints on what frames can be crafted and sent.

Bluetooth Limitations

Primarily BLE — Limited Classic Bluetooth

Most Marauder Bluetooth functions target BLE. Classic Bluetooth (speakers, headphones) has limited support. Do not expect to scan for classic Bluetooth audio devices the way you can with BLE sensors and trackers.

BLE MAC Address Randomization

Modern iOS and Android devices use randomized BLE MAC addresses that change periodically. The same physical device may appear with different MACs across scans. This is an intentional privacy protection built into modern operating systems.

Hardware Quirks

SD Card Write Speed Bottleneck

The ESP32 SPI interface limits SD write speed. During heavy PCAP capture on very busy networks, the write queue can back up and occasionally drop frames. Use a Class 10 / UHS-I card to minimize this.

GPS Cold Start Delay

NEO-6M and similar modules can take 1-5 minutes to acquire a fix from cold start. Warm starts are much faster once almanac data is cached. Leave the GPS module powered outdoors a few minutes before starting a wardriving session if you need a quick fix.

Battery Voltage Sag Under Load

Running Wi-Fi scanning, GPS, display, and SD writes simultaneously draws significant current. This can trigger the ESP32's brownout detector and cause unexpected reboots. If you see frequent resets during heavy use, try a larger battery or reduce display brightness.

Client MAC Address Randomization

Modern smartphones (iOS 14+, Android 10+, Windows 10+) use randomized MAC addresses. Station Scan and Probe Sniff results may show changing MACs from the same physical device across sessions. This is normal operating system behavior.

Recommended Companion Tools & Software

Marauder is most powerful when paired with the right companion software. All tools listed here are free.

Packet Analysis

Wireshark — [wireshark.org](https://www.wireshark.org)

The industry-standard packet analysis tool. Open Marauder's PCAPs and inspect every captured frame in detail. Filter by protocol, follow conversations, decode encryption negotiation. If you only install one companion tool, make it this one. Free — Windows, macOS, Linux.

hcxtools — github.com/ZerBea/hcxtools

Command-line tools for converting Wi-Fi handshake and PMKID capture files into Hashcat-compatible formats. Essential for the authorized credential auditing workflow. Free — Linux (primary), macOS via Homebrew.

Password Auditing (Authorized Use Only)

Hashcat — hashcat.net

The leading open-source, GPU-accelerated password auditing tool. Used in authorized testing to evaluate whether a captured credential hash matches a wordlist. Only use on networks you own or are explicitly authorized to test. Free — Windows, macOS, Linux.

| ⚠ Using Hashcat on credential material from networks you do not own is illegal.

GPS & Mapping

GPS Visualizer — gpsvisualizer.com

Free web tool. Upload your Marauder wardriving CSV and see your route overlaid on satellite imagery. No installation required.

QGIS — qgis.org

Free desktop GIS application for advanced wardriving data visualization and spatial analysis. Overkill for casual use but powerful for detailed coverage mapping. Free — Windows, macOS, Linux.

Device Identification

MAC Vendors — macvendors.com

Web-based OUI lookup. Paste any MAC address to identify the network interface manufacturer. Essential during home network audits for identifying unknown devices in Station Scan results.

Serial Communication

PuTTY (Windows) / minicom (Linux, macOS) / Arduino IDE Serial Monitor

Any of these work for Marauder CLI access at 115200 baud. PuTTY and minicom are lightweight standalone tools. The Arduino IDE Serial Monitor works well if you already have it installed. Set line ending to 'Both NL & CR'.

Changelog & Version Notes

Use this section to track firmware versions, what changed, and hardware-specific observations. Copy the template below for each update.

Version: _____ Date Updated: _____
Hardware: _____ Firmware File Used: _____
Key changes: _____
Issues observed: _____
Notes: _____

 Tip: Keeping a version history saves hours of confusion when a feature changes or disappears after an update — it happens more often than you would think.

Community Resources & Links

The Marauder community is active, welcoming, and the best place to get help, stay updated, and share what you learn.

Official Resources

- GitHub Repository: github.com/justcallmekoko/ESP32Marauder — Source code, firmware releases, hardware docs, issue tracker. The authoritative source for everything Marauder.
- GitHub Releases: github.com/justcallmekoko/ESP32Marauder/releases — All firmware versions with changelogs and hardware-specific binaries. Always download firmware exclusively from here.
- GitHub Wiki: Official documentation covering pinouts, hardware variants, and function references. Linked from the main repository README.

Community Spaces

- Discord: The primary real-time hub. The GitHub README links to the official server. Best for hardware questions, build help, and firmware discussion.
- Reddit: [r/esp32](https://www.reddit.com/r/esp32) and [r/flipperzero](https://www.reddit.com/r/flipperzero) cover Marauder topics regularly. Good for general ESP32 hardware questions and beginner advice.
- YouTube: Search 'ESP32 Marauder' for build guides, feature walkthroughs, and wardriving videos from community creators.

Hardware Sourcing

- Official Store: Check JustCallMeKoko's GitHub profile and community channels for current official device availability.
- AliExpress / Amazon: For DIY components — ESP32 DevKit boards, ILI9341 displays, NEO-6M GPS modules, SD card modules, and LiPo batteries are all widely available.
- Tindie: Marketplace for small-run hardware where community vendors sometimes sell Marauder-compatible builds.

Wardriving Community

- WiGLE: wigle.net — Global wardriving map and statistics platform. Create an account to upload logs and participate in the leaderboards.
- WiGLE Discord: Active wardriving community. Good for route tips, leaderboard discussion, and mapping collaboration.

Bug Reporting

Report firmware bugs on the GitHub Issues page with your firmware version, hardware details, and a clear reproduction description. The more detail you provide, the faster it gets investigated.

Building & Installing Your Own Evil Portal

⚠ Evil Portal is strictly for authorized security awareness demonstrations, captive portal development testing, and closed homelab use. Never deploy a custom portal against real users without their explicit informed consent. Unauthorized use is illegal in virtually every jurisdiction.

The Evil Portal feature lets you serve a fully custom HTML web page to any device that connects to Marauder's rogue access point. This section walks you through building a portal page from scratch, testing it, copying it to your device, and activating it — so you can run professional-looking authorized awareness demonstrations.

How Evil Portal Works

When Evil Portal is active, Marauder does three things simultaneously: it broadcasts an open Wi-Fi access point with any SSID you choose, it runs a DNS server that redirects all domain requests to itself, and it serves your HTML file to any browser that connects. The result is that any device joining the network is immediately shown your custom page, regardless of what URL they try to visit — the same mechanism used by hotels, airports, and coffee shops for their legitimate login portals.

What You Need

- A text editor (Notepad, VS Code, Sublime Text — anything that saves plain text)
- A formatted SD card inserted in your Marauder device
- A card reader to transfer files from your computer to the SD card
- Basic familiarity with HTML is helpful but not required — a template is provided below

File Requirements

- Filename: index.html (exactly — case sensitive on some firmware versions)
- Location on SD card: /portals/index.html
- Format: Standard HTML — no server-side code, no external CDN resources (the device has no internet)
- Size: Keep it under 500KB for reliable serving from the ESP32

⚠ Do not reference external resources like Google Fonts, CDN-hosted JavaScript libraries, or remote images. The rogue AP has no internet connection, so all assets must be embedded directly in the HTML file or stored on the SD card alongside it.

Starter Template: Basic Awareness Training Portal

Copy the HTML below into a file named index.html. This template is styled to look like a generic network login page and is suitable for authorized security awareness demonstrations — it collects nothing and submits nowhere.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Network Login</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f0f2f5;
      display: flex; justify-content: center;
      align-items: center; min-height: 100vh; margin: 0; }
    .box { background: white; padding: 40px; border-radius: 8px;
      box-shadow: 0 2px 12px rgba(0,0,0,0.12); width: 320px; }
    h2 { color: #1a73e8; margin-bottom: 8px; }
    p { color: #555; font-size: 14px; margin-bottom: 20px; }
    input { width: 100%; padding: 10px; margin: 6px 0 14px;
      border: 1px solid #ccc; border-radius: 4px;
      box-sizing: border-box; font-size: 15px; }
    button { width: 100%; padding: 11px; background: #1a73e8;
      color: white; border: none; border-radius: 4px;
      font-size: 15px; cursor: pointer; }
  </style>
</head>
<body>
  <div class="box">
    <h2>Network Access</h2>
    <p>Sign in to continue using this network.</p>
    <input type="text" placeholder="Username or email" />
    <input type="password" placeholder="Password" />
    <button>Connect</button>
  </div>
</body>
</html>
```

```
</div>
</body></html>
```

 **Tip:** This template deliberately does nothing when the button is clicked — it is a visual demonstration only. For an authorized awareness training session, this is intentional: you are showing participants what a phishing portal looks like, not actually capturing anything.

Customising Your Portal

Once you have the basic template working, here are the most impactful things you can customise:

Change the Visual Theme

The entire appearance is controlled by the `<style>` block. Change the background color, box shadow, button color, and fonts to match whatever network or brand you want the portal to simulate for your authorized demonstration. Since all styles are inline, there are no external dependencies to worry about.

Change the SSID (Network Name)

The SSID your rogue AP broadcasts is set in the Marauder Evil Portal menu, not in the HTML file. You can name it anything — 'Airport_Free_WiFi', 'Hotel_Guest', 'CoffeeShop_Open' — to make the demonstration context clear to participants. Choose an SSID that makes sense for your training scenario.

Add a Redirect After Submit

For a more realistic demonstration, you can redirect the user to a real website after they click the button. Add this JavaScript just before `</body>`:

```
<script>
  document.querySelector('button')
    .addEventListener('click', function() {
      window.location.href = 'https://example.com';
    });
</script>
```

Note that since the rogue AP has no real internet connection, the redirect will fail unless the target URL is also hosted locally on the device or the user's device has mobile data as a fallback.

Add Multiple Pages

You can include additional HTML files in the `/portals/` directory and link between them normally with anchor tags. For example, a portal could have an `index.html` login page that links to a `success.html` confirmation page. All files must be placed in `/portals/` and all assets (images, CSS files) must be local — no external URLs.

Embed a Logo or Image

To include an image in your portal, you have two options. The simplest is to copy the image file (e.g. logo.png) into the /portals/ directory alongside index.html, then reference it with a relative path:

```

```

Alternatively, convert your image to a Base64 data URL and embed it directly in the HTML so you only need the single index.html file. Many free online tools can convert an image to Base64.

Installing the Portal on Your Device

- Step 1 — Save your completed HTML as index.html on your computer.
- Step 2 — Remove the SD card from your Marauder device.
- Step 3 — Insert the SD card into your computer using a card reader.
- Step 4 — Navigate to the /portals/ folder on the card. Create it if it does not exist.
- Step 5 — Copy index.html (and any supporting files like images) into /portals/.
- Step 6 — Safely eject the SD card and reinsert it into your Marauder device.
- Step 7 — Power on the device and navigate to Evil Portal in the Marauder menu.
- Step 8 — Select your portal file and set the SSID you want the rogue AP to broadcast.
- Step 9 — Start the Evil Portal. Marauder will begin broadcasting the AP and serving your page.

💡 Tip: Test your portal with your own phone before any demonstration. Connect to the rogue AP and confirm the page loads correctly. Check that fonts, layout, and any images render as expected on a mobile screen.

Testing Your Portal Before Use

You can test your portal page without using Marauder at all — simply open index.html directly in a browser on your computer. This lets you iterate on the design quickly without the SD card transfer cycle every time.

Once you are happy with the design in the browser, do a final test on Marauder itself before your demonstration. Mobile browsers sometimes render things slightly differently, and the ESP32's HTTP server has minor quirks compared to a desktop web server.

Important Limitations

- No HTTPS: Marauder serves pages over plain HTTP. Modern browsers may show a 'Not Secure' warning, which can actually reinforce the awareness training message — real phishing portals often lack HTTPS too.
- No internet: The rogue AP is isolated. Any external links or resources will fail to load.
- Single concurrent connection: The ESP32's HTTP server handles one connection at a time. For group demonstrations, participants should connect one at a time or in quick succession.

- JavaScript limitations: Most standard JavaScript works fine, but complex frameworks (React, Vue, Angular) are overkill and may exceed available memory. Keep your portal's JavaScript simple.
- File size: Keep index.html under 500KB. Embedded Base64 images inflate file size quickly — compress images before embedding.

UI Customisation & Splash Screens

Once you are comfortable with Marauder as a tool, you might want to make the device feel truly yours. The open-source nature of Marauder means the entire codebase is available to modify — and two of the most satisfying personalisation options are custom boot logos and colour theme modifications.

Both require you to compile your own firmware from source rather than flashing a pre-built binary. This is a step up in technical complexity, but it is well within reach for anyone comfortable with the Arduino IDE or PlatformIO, and the community has excellent guides to help.

Custom Boot Logo

When Marauder starts up, it displays a splash screen bitmap before loading the main interface. This image is stored as a C header file containing a raw pixel array, compiled directly into the firmware. Replacing it with your own image — your handle, a logo, or any design you like — is a great first firmware modification project.

The Process

- Step 1 — Create or find your image. For a standard TFT display (ILI9341, 240x320), your splash image should be 240x320 pixels or a proportional subset of that. For smaller OLED displays (128x64), adjust accordingly. Keep it simple — monochrome or limited colour designs convert more reliably.
- Step 2 — Convert the image to a C header file. The tool LCD Image Converter (free, open source) converts a bitmap or PNG into a C array that the firmware can read. Export as a C header with the appropriate colour depth for your display (usually 16-bit RGB565 for TFT displays).
- Step 3 — Clone the Marauder source code from the official GitHub repository.
- Step 4 — Replace the existing splash screen header file in the source with your converted one. The file is typically named something like logo.h or splash.h in the source tree — check the repository README for the exact location.
- Step 5 — Compile and flash using PlatformIO or the Arduino IDE with the ESP32 board support package installed.



Tip: LCD Image Converter is available from sourceforge.net/projects/lcd-image-converter. It handles the pixel format conversion and C array export automatically — you do not need to write any code for this step, just configure the output format to match your display.

The Runaque Splash

If you want to brand your device with your handle, a clean approach is white text on a black background — 'RUNAQUE' in a bold pixel font centred on the screen, with a simple geometric border. This converts cleanly to a 1-bit or 16-bit C array and looks sharp on both TFT and OLED displays. Simple designs also reduce the header file size, keeping your firmware binary lean.

Colour Theme Modification

Marauder's UI colours are defined in the source code, typically in a header or UI source file containing colour constants. By editing these values you can create a completely different visual theme — a dark high-contrast mode for outdoor visibility, a softer low-brightness theme for night use, or any colour scheme that suits your taste.

Finding the Colour Definitions

In the Marauder source code, look for a file named something like `UserInterface.cpp` or a colours header file. Search for `TFT_` prefixed colour constants or hex colour values in the format `0xRRGGBB` or `RGB565` format (`0xF800` for red, `0x07E0` for green, `0x001F` for blue in 16-bit `RGB565`).

Common TFT library colour constants you will encounter include `TFT_BLACK`, `TFT_WHITE`, `TFT_GREY`, `TFT_BLUE`, `TFT_GREEN`, `TFT_RED` and their hex equivalents. Replacing these with your preferred values and recompiling produces a custom theme.

Suggested Theme Ideas

- High Contrast Outdoor: White text on black background, bright yellow highlights. Maximises readability in direct sunlight.
- Night Mode: Dim red text on black background. Red light preserves night vision better than white or blue and is less visible at a distance.
- Amber Terminal: Orange-amber text on black, mimicking classic CRT terminal aesthetics. Easy on the eyes for long sessions.
- Runaque Custom: Whatever colours represent your personal style — this is your device.

💡 Tip: If you are new to firmware compilation, start with just the colour change before attempting the splash screen modification. A colour theme change is a minimal edit — a handful of hex values — and is an excellent first step into source-level Marauder customisation with a very satisfying visible result.

Setting Up the Build Environment

To compile Marauder from source you will need:

- Visual Studio Code with the PlatformIO IDE extension (recommended) — or the Arduino IDE with ESP32 board support
- The Marauder source code cloned from the official GitHub repository
- The correct PlatformIO environment selected for your hardware variant (defined in `platformio.ini`)

- USB serial access to your device for flashing the compiled binary

The official Marauder GitHub repository includes build instructions in the README. The PlatformIO route is generally smoother than the Arduino IDE for this project — it handles library dependencies automatically and the build environments are pre-configured per hardware variant.

⚠ Only compile and flash firmware modifications on your own device. Building from a forked or modified source means your binary is not the same as the official release — document your changes clearly so you can reproduce them after future official updates.

This guide started as a personal reference and grew into something I hope genuinely helps newcomers find their footing with ESP32 Marauder. The device is extraordinary for its size and price — but like most powerful tools, it rewards people who take the time to understand it properly.

If you find errors, have suggestions, or want to contribute updates as the firmware evolves, reach out. The community is what makes projects like this worthwhile.

Use it responsibly. Learn constantly. Have fun.

— Runaque

Made in Antwerp

github.com/Runaque